



Republic of Zambia

Ministry of Education

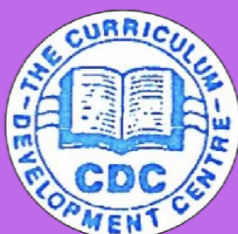
DIRECTORATE OF CURRICULUM DEVELOPMENT

COMPUTER SCIENCE

TEACHING MODULE

FORM 2

Term 1 and 2



Developed by the Curriculum Development Centre

LUSAKA

2026

COPYRIGHT

All rights reserved. No parts of this publication may be reproduced, stored in a retrieval system or transmitted by electronic, mechanical, photocopying, recording or other means, without the prior permission of the Curriculum Development Centre

© Curriculum Development Centre 2026.

List of Authors

The Ministry of Education wishes to express its heartfelt gratitude to the following individuals for their tireless efforts and valuable contributions to the development of the Computer Science Form 2 Teaching Module:

Special recognition is also extended to the following authors for their expertise and contributions:

Lameck Nyirongo	<i>Training Officer – Computer Science and ICT - Directorate of National Science Centre, Lusaka</i>
Garvin Mugala	<i>Subject Teacher - Chizongwe Provincial STEM school, Chipata</i>
James Cosmas Mumba	<i>Subject Teacher - Kapiri Girls National STEM school, Kapiri Mposhi</i>

COORDINATORS

Misheck Mutuzana	<i>Curriculum Specialist -Computer Studies and ICT – Curriculum Development Centre- MoE</i>
Muungu Mabbola	<i>Principal Curriculum Specialist – Technology and Entrepreneurship - Curriculum Development Centre- MoE</i>

The Ministry appreciates the commitment and professionalism demonstrated by the team in ensuring the success of this project.

Table of Contents

COPYRIGHT	2
List of Authors	3
VISION.....	5
Preface.....	6
Acknowledgement.....	7
Guidelines for Teachers on Using Computer Science Module	8
TERM ONE	11
TOPIC 2.1: PRODUCTIVITY TOOLS.....	12
Sub-Topic 2.1.1: Spreadsheets	13
TOPIC 2.2: DATA REPRESENTATION	26
Sub-Topic 2.2.1: Number Systems.....	27
TOPIC 2.3: INTRODUCTION TO PROGRAMMING.....	33
Sub-Topic 2.3.1: Block-Based Programming.....	34
TOPIC 2.4: NETWORKING	42
Sub-Topic 2.4.1: Network Topologies	42
TERM TWO	51
TOPIC 2.5: CYBER SECURITY.....	52
Sub-Topic 2.5.1: Introduction to Cyber Security	53
TOPIC 2.6: ALGORITHMS	59
Sub-Topic 2.6.1: Fundamentals of Algorithms	60
TOPIC 2.7: WEB DEVELOPMENT	68
Sub-Topic 2.7.1: Cascading Style Sheets (CSS) Concepts	69
TOPIC 2.8: DATABASES	78
Sub-Topic 2.8.1: Database Design and Creation	79
REFERENCES	87
APPENDIX: LESSON PLAN.....	90

VISION

Quality, lifelong education for all which is accessible, inclusive and relevant to individual, national and global needs and value systems.

Preface

The introduction of the competence-based curriculum (CBC) marks a transformative step in improving education quality and relevance. This Form 2 Computer Science Teaching Module has been meticulously designed to support teachers in effectively delivering Computer Science lessons under the new curriculum. The module aims to bridge the gap created by the absence of approved textbooks and to equip educators with practical tools, activities, and assessments tailored to the learners' level and needs.

As a Ministry, we understand the challenges that come with transitioning to a new curriculum framework. This module reflects our commitment to making this journey as smooth as possible for teachers, by providing structured guidance, suggested activities, and formative assessments that align with the CBC's objectives. However, we also encourage educators to supplement this module with their own research and innovations to address any gaps that may arise during implementation.

The suggested activities and assessments provided in this module are designed to foster practical and critical thinking skills in learners while ensuring relevance and adaptability to different school contexts. Teachers are encouraged to tailor these suggestions to their unique classroom dynamics and to explore alternative, acceptable approaches when necessary. This flexibility is key to ensuring that all learners, regardless of their environment, benefit from meaningful and impactful Computer Science education.

We hope this module will serve as a valuable resource for teachers as they navigate the new competence-based curriculum, enhancing their ability to inspire and equip learners with essential Computer Science knowledge and skills. Together, let us work towards creating a vibrant and responsive education system that prepares our learners for the demands of the digital age.



Kelvin Mambwe (PhD.)

Permanent Secretary- Educational Services
MINISTRY OF EDUCATION

Acknowledgement

The rapid evolution of technology has transformed every aspect of our lives, and education is no exception. In today's digital age, it is essential that learners acquire computer science knowledge to succeed in the future. This Computer Science Module for Form 2 has been designed as a practical teaching resource for educators, providing clear guidance, lesson plans, and activities to help learners develop foundational skills, critical thinking, and practical expertise in an increasingly digital world.

Aligned with the latest curriculum standards, the module presents real-life scenarios and hands-on exercises to support effective teaching and learning. It has been developed through a consultative and participatory process, ensuring that it is both relevant and user-friendly for teachers. Covering topics from basic computer skills to digital financial tools and creative problem-solving with technology, this module equips teachers to bridge the gap between theoretical knowledge and practical application for their learners.

We hope this module will empower teachers to guide learners in understanding computer science concepts while inspiring them to explore the subject's limitless possibilities in everyday life. Our sincere gratitude goes to the educators and contributors who invested their time and expertise in developing this comprehensive, teacher-focused resource.

We also thank all stakeholders and the Ministry of Education for their financial and technical support, and the staff at the Directorate of Curriculum Development for their dedication in ensuring the successful completion of this teaching module.

Together, let us equip teachers with the tools they need to prepare the next generation for a digital future.



Charles Ndakala (Dr.)

Director - Curriculum Development
MINISTRY OF EDUCATION

Guidelines for Teachers on Using Computer Science Module

The Computer Science modules for Forms 1 to 4 are designed to equip learners with foundational knowledge of how computers work, programming skills, and problem-solving abilities. In the absence of approved textbooks, these modules will serve as comprehensive teaching resources for delivering impactful Computer Science lessons.

1. Understand the Nature of Computer Science as a Subject

Computer Science focuses on:

- The science behind computers, including understanding hardware, software, and networks.
- Developing computational thinking skills to analyse and solve problems.
- Writing and debugging programs in programming languages like Python or Scratch.
- Exploring how data is stored, processed, and managed.

As a science, it goes beyond just using tools—it explains how those tools work and empowers learners to create solutions.

2. Focus on Developing Computational Thinking

The modules emphasise logical reasoning and problem-solving skills. Teachers **should**:

- Start by introducing problem-solving techniques, such as breaking down a problem into smaller steps (decomposition).
- Use examples like designing algorithms or flowcharts to help learners visualise solutions.
- Encourage learners to explain their thought processes when solving programming problems.

3. Teach Programming Through Practice

Programming is a hands-on skill that requires regular practice. To teach programming:

- Use the modules to introduce programming fundamentals, such as variables, loops, and decision-making structures.
- Guide learners to write simple programs, such as a calculator, using the provided examples in the modules.
- Introduce platforms like Python, Scratch, or Code.org for interactive coding sessions.
- Encourage learners to debug their code and explore why errors occur. This reinforces problem-solving skills.

4. Use Scenarios to Build Competences

The module provides hands-on scenarios to help learners develop both understanding and practical skill. Under the CBC, scenarios should be purposeful and contextually rich — placing learners in realistic situations that challenge them to think critically and apply knowledge to solve genuine problems. Teachers should therefore:

- Present scenarios that reflect real-life contexts familiar to learners. Rather than abstract tasks, ground learning in meaningful local situations — for example, asking learners to design a flowchart for how a market trader in Lusaka calculates daily profit, or to write an algorithm that helps a clinic in Chipata manage its patient queue. This ensures learners can transfer knowledge to situations beyond the classroom, which is the core goal of competence-based learning.
- Use scenarios as assessment opportunities by evaluating the process and reasoning demonstrated, not only the final outcome. A learner who works through an imperfect solution but demonstrates clear logical thinking is showing genuine competence development. Provide constructive feedback that identifies what the learner did well and what specific aspect needs refinement.
- Encourage learners to develop their own scenarios for peers to work through. When a learner can construct a realistic problem and present it to others, it demonstrates mastery at the highest level of the CBC competence framework — moving from *knowing* to *applying* to *creating*.
- Connect scenarios to real-world applications so that learners clearly understand the purpose behind what they are learning. Situations such as building a simple app for a school tuck shop, writing a program to automate a repetitive community task, or designing a database for a local small business give learner a compelling reason to engage and help them see computing as a practical life skill.

5. Relate Concepts to Real-World Applications

Learners grasp concepts better when they understand their relevance. For example:

- Explain how databases are used in everyday life, such as managing inventory in a store.
- Discuss how programming is used to develop games, apps, or websites.
- Highlight how artificial intelligence and robotics rely on programming skills.

6. Foster Collaboration and Problem-Solving

Collaborative learning builds confidence and teamwork. Teachers can:

- Organise coding challenges where learners solve problems in pairs or groups.
- Assign group projects, such as designing a game or creating a data management program.
- Encourage learners to share their approaches to problem-solving and learn from one another.

7. Focus on Assessment and Feedback

Assessment in Computer Science should test both theoretical understanding and practical skills. Teachers should:

- Use quizzes to evaluate learners' knowledge of hardware, software, and algorithms.
- Assign coding tasks and grade learners based on functionality, efficiency, and creativity of their programs.
- Provide constructive feedback, pointing out errors in logic and suggesting improvements.

8. Promote Innovation and Critical Thinking

Computer Science encourages learners to think creatively and solve real-world problems. Teachers should:

- Inspire learners to come up with innovative solutions, such as writing a program to calculate household budgets or create a chatbot.
- Use examples in the modules to discuss advancements in technology, such as machine learning and blockchain.

9. Integrate ICT Where Necessary

While Computer Science focuses on programming and system understanding, ICT tools can enhance teaching. For example:

- Use presentation software to explain complex concepts like data flow or computer architecture.
- Incorporate online coding platforms to give learners more exposure to coding environments.

TERM ONE

TOPIC 2.1: PRODUCTIVITY TOOLS

Overview

This topic introduces learners to essential productivity tools, with a primary focus on spreadsheets. Learners will understand how software applications enhance efficiency in organising, calculating, analysing, and presenting data across various contexts such as schools, offices, and homes. By the end of this topic, learners will be able to perform fundamental data entry, analysis, and visualisation tasks.

General Competence(s):

This topic is intended to achieve the following competences among learners:

Analytical Thinking	Analyzing data, identifying trends and patterns, interpreting results from sorting, filtering, and statistical analysis.
Application Software Skills	Performing calculations, creating charts, managing data, and leveraging spreadsheet features.
Collaboration and Communication	Sharing spreadsheets via cloud tools and working on collaborative projects.
Creativity and Innovation	Visualizing data through charts and designing spreadsheets for practical problem-solving.
Critical Thinking and Problem-Solving	Applying formulas, functions, macros, and “what-if” scenarios to solve practical problems.
Digital Literacy	Navigating spreadsheet interfaces, using digital tools effectively, protecting workbooks, and applying formulas.
Financial Literacy	Developing financial models and analyzing “what-if” scenarios.
Technical Proficiency	Using advanced spreadsheet features such as macros, formulas, and data management tools.

Hook

“How do shops calculate totals and profits quickly?” “How do teachers record and analyse learners’ marks efficiently?” Let Learners discuss real-life situations where organising and calculating data accurately is crucial. (You may Pose a follow-up question: “What happens if a single number in a long list of sales is wrong? How does a spreadsheet help find and fix that?”)

Key Terms

Productivity Tools: Software applications that help people create, edit, and manage documents, data, and presentations to work more efficiently (e.g., word processors, spreadsheets).

Spreadsheet: A digital tool made up of rows and columns used to organise, calculate, and analyse data. Think of it as a powerful, digital accountant’s ledger.

Cell: The intersection of a row and a column where data is entered. Each cell has a unique address, like B4.

Formula: A user-defined equation (starting with =) that performs calculations on values in a spreadsheet. Updates automatically if source data changes.

Function: A pre-built tool in a spreadsheet that performs a specific calculation (e.g., SUM, AVERAGE). Functions are like shortcuts for complex calculations.

Chart: A graphical representation of data used to visualise information and reveal patterns or trends at a glance.

Macro: A set of automated instructions that performs a sequence of tasks to save time on repetitive actions.

Sub-Topic 2.1.1: Spreadsheets

Introduction: This sub-topic introduces spreadsheets and their practical applications for organizing and analyzing data. Teachers should guide learners to understand the key components of the spreadsheet interface and explore how spreadsheets can be used effectively in everyday tasks.

Learners should practise performing arithmetic operations using basic formulas and functions, and use filtering and sorting to identify trends, patterns, or specific values. Teach learners how to perform statistical analysis with functions such as COUNTIF, AVERAGE, and MEDIAN, and how to create charts to visualise data clearly. Emphasise data protection by showing how to protect cells, sheets, and workbooks. Guide learners to use collaborative cloud-based tools to share spreadsheets, create and apply macros, and develop financial models using what-if scenarios. Include practical exercises to help learners apply these skills effectively.

Specific Competence(s) – Learners will:

- 2.1.1.1 Use spreadsheet for data entry and analysis.

Learning Activities

Activity 1: Discussing the Basic Components of the Spreadsheet Interface

Learning Environment Setup

Choose any of the following but not limited to:

Artificial (Physical – Classroom)

- Classroom: Use board or charts to discuss spreadsheet components.
- Charts and Posters: Display labelled spreadsheet parts.
- Hands-On Demo: Learners explore spreadsheet software on computers.

Technology Environment

- Computer Laboratory: Learners interact directly with spreadsheet tools.
- Projector/Digital Display: Demonstrate spreadsheet interface live.
- Videos/Tutorials: Use clips to explain components and functions.

Natural Environment

- Real-Life Observation: Learners observe spreadsheet use in offices or schools

Teaching and Learning Materials:

Choose any of the following but not limited to:

- Computers with spreadsheet software (Microsoft Excel, LibreOffice Calc, or Google Sheets)
- Printed screenshots of the spreadsheet interface for learners without computer access
- Projector or large display for teacher demonstration

Suggested Teaching Methods:

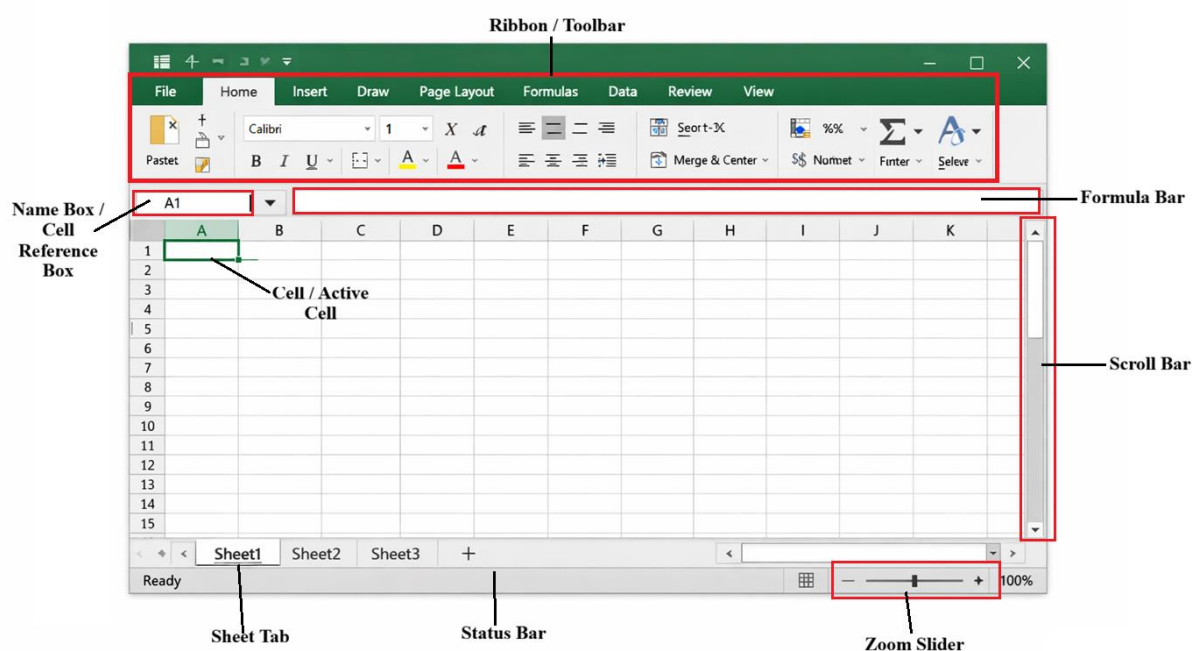
- **Collaborative Learning:** Group interaction to identify spreadsheet components and understand their functions
- **Demonstration Method:** Showing how spreadsheet components and functions operate.
- **Discussion Method:** Sharing ideas on how spreadsheets improve accuracy and productivity.
- **Hands-on Activities:** Use of spreadsheets to enter data and apply simple formulas.
- **Inquiry-Based Learning:** Exploration of how spreadsheets calculate totals and reduce errors.
- **Observation Method:** Viewing and identifying spreadsheet components during demonstrations.
- **Project-Based Learning:** Creation of simple marks or sales records using spreadsheets.
- **Question and Answer Method:** Guided questioning to reinforce understanding.
- **Real-World Exposure Method:** Linking spreadsheet use to real-life situations such as school records and business sales.

▪ How to Conduct:

Begin the lesson by projecting a live spreadsheet on the board (or distributing printed screenshots if computers are unavailable). Ask learners to silently study the interface for two minutes, then facilitate a whole-class "scavenger hunt": call out a component name (e.g., "formula bar") and have learners race to identify and click it. Introduce each component – workbook, worksheet, rows, columns, cells, cell address, formula bar, and ribbon – with a brief explanation. Consolidate with a labelled diagram exercise where learners annotate a printed screenshot. Close by asking: "If cell A1 contains 5 and B1 contains 3, what would $=A1+B1$ in C1 show?" to preview the next activity.

Suggested Learning Points

- ◇ Open the spreadsheet
- ◇ Explore the Basic Components of the Spreadsheet Interface



Basic Components of the Spreadsheet Interface

□ **Content Tip:** A spreadsheet is modelled on an accountant's ledger, but its real power lies in the dynamic link between cells. When data in one cell changes, every formula and chart referencing it updates instantly – turning a static table into a live data model. Stress this 'live update' concept early; it is the single most important differentiator from a paper ledger and will motivate learners to master formulas. Also point out that keyboard shortcuts (Ctrl+Home to jump to A1; Ctrl+End to find the last used cell) dramatically speed up navigation in large spreadsheets.

Learning Activities

Activity 2: Exploring the Uses and Benefits of Spreadsheets in Daily Life

Learning Environment Setup

Artificial (Physical – Classroom)

- Classroom discussions on uses and benefits of spreadsheets in daily life.
- Charts and posters showing examples such as budgeting, marks recording, and sales tracking.
- Role play activities simulating simple record-keeping situations.

Technology Environment

- Computer laboratory for exploring spreadsheet software and its features.
- Projector or digital display for demonstrating real-life applications.
- Interactive tutorials showing how spreadsheets support data organisation and calculation.

Natural Environment

- Observation of spreadsheet use in real settings such as shops, schools, and offices.

Teaching and Learning Materials

- Computers with spreadsheet software
- Projector and screen to display and compare platform interfaces
- Charts and posters
- Manila paper and markers for group brainstorming
- Role cards for each group context
- Sample worksheets (marks sheets, sales records)
- Printed screenshots of spreadsheets
- Activity sheets and exercise books

Suggested Teaching Methods:

- **Cooperative Learning:** Learners work in small groups to share ideas and complete the task together.
- **Role-Play / Context-Based Learning:** Learners think and respond based on real-life situations like school, home, or business.
- **Brainstorming:** Learners quickly generate ideas without criticism to encourage creative thinking.
- **Class Discussion:** Learners present and compare ideas while the teacher guides understanding.
- **Demonstration:** The teacher shows features of Microsoft Excel, Google Sheets, and LibreOffice Calc to make learning practical.
- **Experiential Learning:** Learners practice using spreadsheets to gain hands-on experience.
- **Guided Discovery:** The teacher asks questions that help learners discover spreadsheet advantages themselves.
- **Reflective Questioning:** Learners think about and evaluate what they have learned at the end of the activity.

▪ How to Conduct:

Organise learners into groups of four. Each group receives a role card assigning them a context: school (recording exam marks), home (tracking a family budget), business (monitoring shop sales), or community (recording donations for a school fundraiser). Groups brainstorm for five minutes and list three specific tasks their context uses spreadsheets for, plus two advantages over manual calculation. Groups then present to the class, and the teacher compiles a master list on the board, grouping advantages into categories: accuracy, speed, and decision support. Conclude with a brief comparison of platforms: Microsoft Excel (industry standard), Google Sheets (real-time collaboration), and LibreOffice Calc (free and open-source).

□ **Content Tip:** Make the comparison of platforms concrete. Show learners that Google Sheets allows multiple people to edit the same file simultaneously – each user has a coloured cursor and changes appear in real time. This is a powerful selling point for group work. For Excel, highlight its advanced analytical tools (PivotTables, Power Query) relevant for future careers. For LibreOffice Calc, emphasise that it is completely free and works offline – important in contexts where internet access or software licences are limited.

Learning Activities

Activity 3: Performing Arithmetic Operations Using Basic Formulas and Functions

Learning Environment Setup

Artificial (Physical – Classroom)

- Arrange learners in a computer lab with desks facing the instructor's screen.
- Ensure each learner (or pair) has access to a computer.
- Provide clear visibility of the projector/board.
- Maintain a quiet, well-lit, and ventilated space.
- Allow teacher movement for monitoring and support.

Technology Environment

- Computers/laptops with spreadsheet software (Microsoft Excel, Google Sheets).
- Projector or large display for demonstrations.
- Pre-prepared dataset (shop inventory table).
- Reliable power supply or backup.
- Optional: Internet access for sharing or Google Sheets.

Natural Environment

- Encourage questions and experimentation.
- Promote peer collaboration.
- Allow learners to learn from mistakes.
- Use real-life context (shop inventory).

Teaching and Learning Materials:

Choose any of the following but not limited to:

- Computers with spreadsheet software
- Pre-prepared shop inventory dataset (printed or shared as a file)
- Step-by-step formula reference card for learners
- Pre-populated spreadsheet (Item, Unit Price, Quantity, Total)
- Projector/whiteboard
- Instruction sheets (printed/digital)
- Chalk/markers (if needed)
- Practice worksheets
- Sample formulas: `=C2*B2`, MAX, AVERAGE

Suggested Teaching Methods:

- **Demonstration Method:** Show typing `=C2*B2`, explain "=", copying formulas, and absolute reference (`$F$1`).
- **Guided Practice:** Learners enter and copy formulas, apply tasks with teacher support and feedback.
- **Inquiry-Based Learning:** Explore formula changes and relative vs absolute references using guiding questions.
- **Hands-on Learning:** Learners input formulas, use MAX and AVERAGE, and solve problems.
- **Independent Practice:** Calculate totals, find maximum values, and averages.
- **Collaborative Learning:** Learners work in pairs to compare answers and solve errors.

▪ How to Conduct:

Provide learners with a pre-populated dataset: a simple shop inventory table with columns for Item, Unit Price (K), Quantity, and Total. First, demonstrate how to type $=C2*B2$ in the Total column, explaining the $=$ sign as the "trigger" that tells the spreadsheet a formula is coming. Then introduce relative references by copying the formula down the column – show how the row number adjusts automatically. Next, contrast with an absolute reference: add a Discount Rate in cell F1 and modify the formula to $=C2*B2*(1-\$F\$1)$, demonstrating that the $\$$ sign "locks" the reference when copying. Allow learners 15 minutes of guided practice, then set an independent challenge: calculate total stock value, find the most expensive item using MAX, and find the average price using AVERAGE.

□ **Content Tip:** The distinction between relative (A1) and absolute (\$A\$1) references is one of the most common sources of errors for beginners – and one of the most important concepts in spreadsheet literacy. A concrete demonstration is essential: copy a formula with a relative reference sideways and let learners observe how the column letter shifts. Then copy a formula with an absolute reference and let them see it stay fixed. A multiplication table is an ideal demonstration tool: the row headers use an absolute column reference (\$A2) and the column headers use an absolute row reference (A\$1), so a single formula in B2 can be copied to fill the entire table correctly.

Learning Activities

Activity 4: Filtering and Sorting Data to Identify Trends, Patterns, or Specific Values

Learning Environment Setup

Artificial (Physical – Classroom)

- Arrange learners in a computer lab with desks facing the instructor's screen.
- Ensure each learner (or pair) has access to a computer.
- Provide clear visibility of the projector/board.
- Maintain a quiet, well-lit, and ventilated environment.
- Allow space for teacher movement and support.

Technology Environment

- Computers/laptops with spreadsheet software (Microsoft Excel, Google Sheets).
- Projector or digital display for demonstrations.
- Pre-prepared dataset (class marks with 30 learners).
- Reliable power supply and optional internet access.

Natural Environment

- Encourage active participation and questioning.
- Promote collaboration and peer support.
- Allow learners to explore and learn from mistakes.
- Use real-life context (analysis of learners' performance).

Teaching and Learning Materials:

Choose any of the following but not limited to:

- Computers with spreadsheet software
- Pre-prepared class marks dataset
- Printed challenge question cards
- Class marks dataset (names, subject scores, total, grade)
- Computers with spreadsheet software
- Projector/whiteboard
- Instruction sheets or activity tasks

Suggested Teaching Methods:

- **Demonstration Method:** Show how to sort data and apply AutoFilter.
- **Guided Practice:** Learners follow steps to sort and filter with support.
- **Inquiry-Based Learning:** Start with questions like "Who scored highest in Mathematics?"
- **Hands-on Learning:** Learners practice sorting and filtering to answer questions.
- **Problem-Solving Method:** Tasks such as totals above 200 and identifying failures.
- **Collaborative Learning:** Learners work in pairs/groups to compare results.
- **Discussion Method:** Discuss why sorting/filtering is better than manual scanning.
- **Question and Answer Method:** Use progressive questions to guide learning.

How to Conduct:

Provide a class marks dataset (30 rows: learner name, subject scores for three subjects, total, and grade). Begin with an unstructured question: "Which learner scored highest in Mathematics?" Let learners attempt to answer by scanning the list – this surfaces the problem that manual scanning is slow and error-prone. Then demonstrate sorting the data by the Mathematics column (Data > Sort), asking learners to predict the result before executing. Next, introduce AutoFilter: apply filters to show only learners who scored "Distinction" in all three subjects. Pose progressive challenges: "Show only learners whose total is above 200." "How many learners failed Science?" Conclude by having learners remove all filters and explain why the underlying data is unchanged.

□ **Content Tip:** An important conceptual point to reinforce: filtering does NOT delete data – it temporarily hides rows that do not meet the criteria. The data is always there and can be restored by clearing the filter. This prevents panic when learners "lose" rows. Sorting, on the other hand, reorders data permanently (though Ctrl+Z can undo it). A best practice tip for real-world use: always keep an original "ID" or row-number column before sorting, so data can be returned to its original order if needed. Posing questions as problems ("Find all learners who scored above 80% in Biology") rather than instructions ("Apply a filter") builds analytical thinking alongside the technical skill.

Learning Activities**Activity 5: Performing Statistical Analysis Using Functions (COUNTIF, AVERAGE, MEDIAN...)****Learning Environment Setup****Artificial (Physical Classroom):**

- Computer lab or classroom arranged for pair/group work
- Clear view of board/projector

Technology Environment:

- Computers with Microsoft Excel or Google Sheets
- Projector and prepared dataset

Natural Environment:

- Quiet, supportive environment for concentration and discussion

Teaching and Learning Materials

- Computers with spreadsheet software
- Class marks dataset (continuation from Activity 4)
- Function reference card listing syntax for SUM, AVERAGE, MIN, MAX, MEDIAN, COUNTIF
- Projector/board

Suggested Teaching Methods

- **Demonstration:** Teacher shows how functions work
- **Q&A:** Engage learners before introducing formulas
- **Guided Practice:** Learners apply functions step-by-step
- **Group Work:** Learners discuss and compare results
- **Discussion/Analysis:** Learners give observations from data

▪ How to Conduct:

Use the same class marks dataset from Activity 4. Begin with a show of hands: "Who can tell me the average score without a calculator?" – then reveal how `=AVERAGE(D2:D31)` calculates it instantly. Introduce each statistical function in a "function gallery" format: display the function name, its syntax, and a one-line description on the board. Learners then apply each function to the dataset: `=SUM` for total marks in the class, `=MIN` and `=MAX` to identify the lowest and highest scorers, `=MEDIAN` to find the middle value, and `=COUNTIF(E2:E31,"Distinction")` to count grades meeting a criterion.

Conclude with a comparative analysis task: "Using these functions, write three observations about the class performance and suggest one area the teacher should address."

□ **Content Tip:** A function is a pre-built formula that handles complex or repetitive operations – saving time and reducing errors. Familiarise learners with the "Insert Function" button (fx) or the "Function Library" in the ribbon, which categorises all available functions. This empowers independent exploration beyond what is taught in class. The COUNTIF function is particularly powerful and often underused: it counts only the cells that meet a specified condition, making it invaluable for tasks like counting how many learners achieved a certain grade. Distinguishing COUNTIF (counts cells meeting a condition) from SUM (totals values) from AVERAGE (finds the mean) is a critical conceptual checkpoint.

Learning Activities

Activity 6: Creating Charts to Visualise Data

Learning Environment Setup

Artificial (Physical Classroom):

- Computer lab arranged for **individual or pair work**
- Clear view of projector for chart demonstrations

Technology Environment:

- Computers with Microsoft Excel or Google Sheets
- Projector and tuck shop sales dataset

Natural Environment:

- Quiet, interactive environment that supports **discussion and critique**

Teaching and Learning Materials:

- Computers with spreadsheet software
- Tuck shop sales dataset
- Chart type reference guide (when to use bar, pie, line Computers/laptops)
- Projector/interactive board

Suggested Teaching Methods

- **Discussion:** Introduce importance of charts (rainfall example)
- **Demonstration:** Teacher shows how to create bar/column charts
- **Guided Practice:** Learners follow steps to create charts
- **Independent Learning:** Learners create a pie chart on their own
- **Comparative Discussion:** Chart critique (bar vs pie)
- **Question & Answer:** Evaluate which chart best represents data

▪ How to Conduct:

Start with a discussion question: "If I give you a list of 12 monthly rainfall figures, how quickly can you tell me which month was wettest?" Then show the same data as a bar chart – the answer is immediate. Provide learners with a dataset of the tuck shop sales from Topic 2.1's scenario (items and revenue). Guide them step-by-step to select the data, insert a column/bar chart, and add a title and axis labels. Then ask learners to independently create a pie chart from the same data. Run a "chart critique" session: project two charts side-by-side (bar vs. pie of the same data) and ask: "Which chart better shows which item raised the most money? Which better shows each item's share of the total?" This develops critical evaluation alongside technical skill.

□ **Content Tip:** Charts transform rows of numbers into immediate visual insights, but choosing the wrong chart type can mislead the reader. A bar/column chart is best for comparing quantities across categories (e.g., sales per item). A pie chart is best for showing proportions of a whole (e.g., each item's percentage of total sales) – but only when there are fewer than six or seven categories. A line chart is best for showing change over time (e.g., monthly sales trends). Teaching learners to justify their chart choice, not just create it, builds the analytical literacy required at higher levels and in the workplace.

Learning Activities

Activity 7: Protecting Cells, Sheets, and Workbooks

Learning Environment Setup

Artificial (Physical Classroom):

- Computer lab arranged for individual hands-on practice
- Clear visibility of projector for demonstration

Technology Environment:

- Computers with Microsoft Excel or Google Sheets
- Projector and class marks spreadsheet

Natural Environment:

- Quiet, focused environment that supports careful practice and problem-solving

Teaching and Learning Materials:

- Computers with spreadsheet software
- Pre-built budget or marks spreadsheet with formulas
- Step-by-step protection guide handout
- Spreadsheet with formulas (class marks dataset)
- Projector/board

Suggested Teaching Methods

- **Discussion:** Use real-life scenario to introduce need for protection
- **Demonstration:** Show locking/unlocking cells and protecting sheet
- **Guided Practice:** Learners follow steps to protect cells
- **Hands-on Practice:** Learners apply protection independently
- **Q&A:** Check understanding of why protection is important

▪ How to Conduct:

Present a scenario: "You have spent two hours building a budget spreadsheet with 20 formulas. You share it with a colleague who accidentally types over one formula cell. The whole spreadsheet now shows wrong totals, and you don't know which cell was changed." Ask learners: "How could this have been prevented?" Introduce the concept of cell protection as the solution. Demonstrate a two-step process: first, select only the input cells (cells where users should type) and uncheck "Locked" in Format Cells. Then, protect the sheet with a password (Review > Protect Sheet). Show learners that the formula cells are now locked, but the input cells remain editable. Allow learners to practise on the class marks spreadsheet, locking all formula cells and leaving only the score-entry cells open.

□ **Content Tip:** Protection in spreadsheets works in a counterintuitive way: by default, ALL cells in a spreadsheet are marked as 'Locked', but this has no effect until sheet protection is turned on. The correct workflow is: (1) select the cells you want to remain editable, (2) uncheck 'Locked' for those cells, (3) then enable sheet protection. Understanding this two-step logic is essential. A good classroom discussion prompt: "Why would you want to protect a spreadsheet before sharing it with a classmate for peer review?" The answer – to prevent accidental formula breakage – reinforces data integrity as a professional habit, not just a technical option.

Learning Activities

Activity 8: Sharing Spreadsheets Using Collaborative (Cloud-Based) Tools

Learning Environment Setup

Artificial (Physical Classroom):

- Computer lab arranged for group or pair collaboration
- Seating allows learners to interact and observe shared work

Technology Environment:

- Computers with internet access
- Google Sheets (or similar cloud tool)
- Projector for live demonstration

Natural Environment:

- Interactive and cooperative environment that encourages sharing and teamwork

Teaching and Learning Materials

- Computers with internet access and Google Sheets (or Microsoft 365 OneDrive)
- Internet connection (if available)
- Pre-made budget/expense tracker spreadsheet with at least two formulas
- Projector/board
- Role-play cards (for offline simulation)
- A Google or Microsoft account for the teacher to share from

Suggested Teaching Methods

- **Discussion:** Introduce problem of multiple file versions
- **Demonstration:** Show real-time collaboration and sharing
- **Guided Practice:** Learners practise sharing and editing
- **Role Play/Simulation:** For offline collaboration understanding
- **Collaborative Learning:** Learners work together on one file
- **Q&A:** Discuss permission levels (Viewer, Commenter, Editor)

▪ How to Conduct:

Open with a class discussion: "Raise your hand if you have ever experienced this: you email a file to someone, they edit it, email it back, but by then you have also made changes to your copy – and now you have two different versions." Validate the frustration and explain that cloud-based collaboration solves this version-control problem. If internet is available, demonstrate Google Sheets: share the pre-made budget spreadsheet with several learners simultaneously and show how their cursors and changes appear in real time. If internet is unavailable, simulate the process by assigning roles ("you are User A, you are User B") and discussing what each user would see. Discuss sharing permission levels (Viewer, Commenter, Editor) and when each is appropriate.

□ **Content Tip:** Cloud-based spreadsheets solve the version-control problem because everyone works on a single, shared live file – there is no "Final_v3_actuallyfinal.xlsx" problem. Key features to highlight for Google Sheets: the version history (File > Version History > See Version History) allows you to see every change ever made and who made it, and to restore a previous version. This is an important accountability and recovery tool. For Microsoft 365, OneDrive provides the same collaborative capability. Reinforce the protection tip from Activity 7: always protect formula cells before sharing, even with collaborators, so the model's logic remains intact while allowing data input in the designated cells.

Learning Activities

Activity 9: Creating and Applying Macros

Learning Environment Setup

Artificial (Physical Classroom):

- Computer lab arranged for individual hands-on practice
- Clear view of projector for macro demonstration

Technology Environment:

- Computers with Microsoft Excel
- Projector for live demonstration, Sample spreadsheet for practice

Natural Environment:

- Focused and controlled environment to support step-by-step learning and accuracy

Teaching and Learning Materials

- Computers with Microsoft Excel (which has the most comprehensive macro support)
- Spreadsheet (blank and sample marks report), Projector/board
- Pre-prepared blank report template for the macro demonstration

Suggested Teaching Methods

- **Demonstration:** Show manual task vs macro automation
- **Guided Practice:** Learners record and run a macro
- **Hands-on Learning:** Learners practise independently
- **Discussion:** Highlight efficiency and real-life use
- **Safety Awareness:** Explain risks of macro-enabled files

▪ How to Conduct:

Select a repetitive formatting task that learners can relate to: preparing a monthly marks report (setting column widths, making the header row bold and purple, adding cell borders, and freezing the top row). Ask for a volunteer and time them doing this manually on a blank spreadsheet – it typically takes 3–5 minutes. Then demonstrate recording a macro that does the same task: Tools/View > Macros > Record Macro, perform the formatting steps, stop recording. Name the macro "FormatReport" and assign it to a button. Click the button on a fresh sheet and show the formatting applied in under one second. Allow learners to record their own simple macro. Close with a safety discussion: macro-enabled files (.xlsm) can carry harmful code, so macros should only be enabled from trusted sources.

□ **Content Tip:** Macros automate repetitive sequences of actions, saving significant time in real-world workflows. The Record Macro feature is the ideal entry point for beginners – it captures actions visually without requiring knowledge of VBA (Visual Basic for Applications), the underlying programming language. Do not attempt to teach VBA code at this stage; the goal is the concept of automation, not the syntax of a programming language. Practical uses learners will encounter: automatically formatting a monthly report, cleaning and standardising imported data, or generating a consistent output from a template. Always emphasise the security warning: files with macros must come from trusted sources, as malicious macros are a real cyber threat (connecting to Topic 2.5).

Learning Activities

Activity 10: Developing Financial Models Using What-If Scenarios

Learning Environment Setup

Artificial (Physical Classroom):

- Computer lab arranged for **group collaboration**
- Clear view of board/projector for modelling steps

Technology Environment:

- Computers with Microsoft Excel
- Projector for demonstrating Scenario Manager
- Pre-designed or blank financial model template

Natural Environment:

- Interactive and analytical environment that supports discussion and decision-making

Teaching and Learning Materials

- Computers with spreadsheet software
- Financial data/model template
- Projector/board
- Step-by-step guide for Scenario Manager
- Partially built three-part budget model template
- Scenario planning worksheet with the three cases defined

Suggested Teaching Methods

- **Discussion:** Introduce real-life financial scenario
- **Demonstration:** Show model structure and Scenario Manager
- **Guided Practice:** Learners build the model step-by-step
- **Collaborative Learning:** Groups analyse different scenarios
- **Problem-Solving:** Compare Best, Base, Worst cases
- **Q&A:** Interpret results and make decisions

How to Conduct:

Frame this activity as a real-world simulation: "You are a financial advisor. A client wants to know: if I increase my monthly savings by K200, how much more will I have in five years? What if the bank interest rate changes?" Introduce the three-part model structure on the board: (1) Inputs/Assumptions – a clearly labelled section for variables (monthly income, rent, interest rate, salary increase); (2) Calculations – the main working area where formulas reference the Inputs section; (3) Outputs/Results – a summary showing key outcomes (total savings, disposable income). Have learners build this model collaboratively. Then demonstrate Scenario Manager (Data > What-If Analysis > Scenario Manager) to create and compare a Best Case (10% salary increase), Base Case (3%), and Worst Case (no increase) scenario. Run a Scenario Summary report and discuss the results.

□ **Content Tip:** The separation of inputs from calculations from outputs is the hallmark of professional financial modelling. This structure makes the model easy to audit, update, and share: anyone can change the input values and immediately see the impact on the outputs, without needing to understand the formulas. There are two complementary What-If tools: Scenario Manager is best for comparing a small number of distinct, pre-defined situations (e.g., optimistic vs. pessimistic). Data Tables (Data > What-If Analysis > Data Table) are better for seeing a continuous range of outcomes driven by one or two variables (e.g., how do total savings change if the interest rate varies from 1% to 15%?). Always protect the calculation and output cells of a model before sharing, so that collaborators can only modify the input cells and cannot accidentally break the underlying logic.

Expected Standard

Entered and analysed data correctly using spreadsheet.

ASSESSMENT

Examples of Formative

Scenario: Your cousin Bwalya runs a small kantemba at Soweto Market in Lusaka. She sells vegetables such as tomatoes, onions, and cabbage. She has an exercise book where she writes each day's sales, but at the end of the week, she spends hours adding everything with a calculator and often makes mistakes. She has heard that a computer can help her.

1. (Data Entry & Interface) You open a blank spreadsheet for Bwalya. She looks at all the empty boxes and says, "I'm tired, where do I even start?" Explain to her in simple terms what a cell is and how she should organise her vegetables, quantities sold, and prices into rows and columns.
2. (Formula & Function) After a week of typing in data with your help, Bwalya says, "Now I have to add all these figures again for my records!" What advice would you give her about using a formula or function so the computer does the adding automatically every time she enters new data?

Formative Rubric:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Data Entry & Interface Navigation	Proficiently navigates all interface parts; enters data with 100% accuracy using shortcuts.	Correctly identifies main interface components; enters data with minimal errors.	Identifies some components; data entry has frequent errors requiring correction.	Struggles to navigate; data entry is disorganised and highly inaccurate.
Formula & Function Application	Independently creates complex formulas with mixed references and uses functions effectively.	Creates basic arithmetic formulas and uses common functions (SUM, AVG) with teacher guidance.	Creates simple formulas but makes frequent errors; requires step-by-step help.	Unable to create a working formula; cannot identify or use a function.
Data Organisation (Sort/Filter)	Applies sorting and filtering to analyse data and answer complex questions independently.	Correctly sorts and filters data as directed to find specific information.	Attempts to sort/filter but often selects wrong data range or criteria.	Does not understand the purpose of sorting/filtering or how to apply them.
Chart Creation	Selects and creates the most effective chart type independently; customises for clarity.	Creates a correct basic chart (column, pie) from selected data with some guidance.	Creates a chart but with significant errors (e.g., wrong data, chart type).	Cannot create a chart.
Terminology Knowledge	Accurately uses all key terms in context during discussions and quizzes.	Can define most key terms correctly in a quiz.	Confuses several key terms (e.g., formula vs. function).	Cannot define basic terms like cell, row, or column.

Example of Summative Assessment:

Scenario: Your school's Parent-Teacher Association (PTA) in Ndola is organising a fundraising fun day. You have been asked to manage the finances for the tuck shop, which will sell snacks, drinks, and fritters.

Task:

1. Create a spreadsheet to track the tuck shop sales. Include columns for: Item, Quantity Sold, Price Per Item (Kwacha), and Total (Kwacha).
2. Use a formula to automatically calculate the "Total" for each item (Quantity Sold $\times$ Price Per Item).
3. Use a function at the bottom of the "Total" column to calculate the overall money raised for the school.
4. Create a simple chart (e.g., a bar graph) that shows how much money each item brought in.
5. Write one sentence in Bemba, Nyanja, or English explaining which item was the most profitable and why this information is useful for planning the next event.

Summative Rubric:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Data Organisation & Entry	Data is logically structured with clear labels and formatting; no entry errors.	Data is well-organised with appropriate labels; minor, non-critical errors.	Data is entered but poorly organised; several errors.	Data is disorganised, unlabeled, and contains critical errors.
Formula & Function Use	Uses a variety of appropriate functions and complex formulas correctly, including mixed references.	Uses correct basic formulas (+, -, *, /) and key functions (SUM, AVERAGE).	Uses some formulas, but they contain logical or reference errors.	No formulas or functions used; all calculations are manual.
Data Visualisation	Creates a well-formatted, accurate chart that best represents the data and enhances understanding.	Creates a correct and appropriately chosen chart for the data.	Creates a chart, but it is inaccurate or poorly chosen for the data.	No chart is created, or the chart is completely wrong.
Interpretation & Analysis	Provides a thorough, insightful written analysis of what the data and chart reveal.	Provides a basic, correct written summary of the data.	Interpretation is vague, incorrect, or missing key points.	No written interpretation is provided.
Overall Functionality	Spreadsheet is a dynamic, interactive model; updates to source data flow through formulas and chart correctly.	Spreadsheet functions correctly for the given task.	Spreadsheet works partially but has errors that prevent full functionality.	Spreadsheet is non-functional.

Summary**Key Points Recap:**

- Spreadsheets are powerful tools for organising, calculating, and analysing numerical data.
- Core skills include data entry, using formulas/functions, sorting/filtering, and creating charts.
- Spreadsheets improve accuracy, save time, and aid in decision-making across many aspects of life and work.
- Understanding how to protect and share data is an important part of digital literacy.

TOPIC 2.2: DATA REPRESENTATION

Overview

Data representation refers to the way data such as numbers, text, images, and sound are stored and processed in a computer. Since computers only understand binary, different number systems and coding schemes are used. This topic introduces learners to binary, denary, octal, and hexadecimal number systems, and how text is represented using coding systems like ASCII.

General Competence(s):

Analytical Thinking	Analyses patterns and relationships when converting between number systems.
Application Software Skills	Uses calculators or spreadsheet tools to verify conversions and operations.
Collaboration and Communication	Works in groups to discuss concepts and explain conversion processes.
Creativity and Innovation	Finds different methods or shortcuts to convert and represent data.
Critical Thinking and Problem-Solving	Solves conversion problems and performs number system operations step-by-step.
Data Structure Skills	Understands how data (numbers and characters) is structured in computers.
Digital Literacy	Understands how computers represent data using number systems and ASCII.
Technical Proficiency	Demonstrates understanding of how digital systems process and store data.

Hook

The teacher writes "HELLO" on the board and asks: "Do you know that this word is stored in a computer as numbers?" The teacher then writes the binary for each letter: H←01001000, E←01000101, L←01001100, L←01001100, O←01001111. Learners discuss how letters, numbers, and images can all be represented using just 0s and 1s. (Ask: "If computers only understand 0 and 1, how can I play a colourful video game or see a photo?")

Key Terms

- **Data Representation:** The method used internally by a computer to store and process different forms of data using codes.
- **Number System:** A writing system for expressing numbers, defined by its base (e.g., binary, denary).
- **Binary:** A base-2 number system using only 0 and 1 – the fundamental language of computers, representing off (0) and on (1) states of electricity.
- **Denary/Decimal:** The base-10 number system used in everyday life, using digits 0 through 9.
- **Octal:** A base-8 number system using digits 0 through 7. Used in older computer systems and for Unix/Linux file permissions.
- **Hexadecimal:** A base-16 number system using digits 0–9 and letters A–F. One hex digit represents exactly four binary bits.
- **ASCII (American Standard Code for Information Interchange):** A coding standard that assigns a unique number (0–127) to represent text characters in computers.
- **Bit:** The smallest unit of data in a computer – a single binary value, either 0 or 1.
- **Byte:** A unit of digital information consisting of 8 bits – enough to represent a single character.

Sub-Topic 2.2.1: Number Systems

Introduction: This sub-topic introduces number systems as a way of writing and using numbers. The most common system is the decimal system, which uses digits from 0 to 9. In computing, other number systems such as binary, octal, and hexadecimal are used to represent and process data. Understanding number systems helps you see how numbers are used in everyday life and how computers store and handle information.

Specific Competence(s) – Learners will:

- 2.2.1.1 Apply Number System Operations

Learning Activities

Activity 1: Discussing Concepts of Hexadecimal and Octal Number Systems

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for group work with space to create posters.
- Clear view of board/projector for class discussion and comparison table.

Technology Environment:

- Optional: computers or tablets for reference or digital base charts.
- Projector/board to display examples of number systems.

Natural Environment:

- Interactive and collaborative environment that encourages discussion, creativity, and presentation.

Teaching and Learning Materials

- Base Chart templates (Binary, Denary, Octal, Hexadecimal)
- Manila paper, markers, and pens for posters
- Projector/board, Sample memory address examples
- Screenshots of a Linux `chmod` command and a web colour picker (for context)

Suggested Teaching Methods

- **Discussion:** Introduce number system concepts using analogies.
- **Group Work/Collaboration:** Groups complete charts and create posters.
- **Presentation:** Groups explain their charts to the class.
- **Question and Answer:** Facilitate critical thinking about why hexadecimal is preferred over binary.
- **Demonstration/Consolidation:** Fill class comparison table and summarise key points

How to Conduct:

Begin with a brief analogy to activate prior knowledge: "We use a base-10 system because humans have ten fingers. A clock uses base-60 for seconds. Computers use base-2 because their switches have only two states: off and on." From this, build upward. Organise learners into groups and distribute a "Base Chart" template with four columns (Binary, Denary, Octal, Hexadecimal). Groups list the valid digits for each system and calculate its base. Each group then creates a large manila-paper poster of their chart and presents it, explaining why the system exists. Consolidate by filling a class comparison table on the board and asking: "Why would a programmer prefer hexadecimal over binary when writing a memory address?" – leading to the answer that one hex digit replaces four binary digits, making it far more compact.

□ **Content Tip:** Connect each number system to a tangible real-world use to make them memorable. Binary (base-2): every electronic switch, from a transistor to a hard drive, uses two states. Octal (base-8): used in Unix/Linux file permission codes – e.g., `chmod 755` sets read, write, execute permissions using octal digits. Hexadecimal (base-16): used everywhere humans need to read binary data compactly – memory addresses (`0x00FF`), colour codes in web design (`#FF0000` is pure red), and MAC addresses on network devices. Showing a real Linux terminal running `chmod` and a CSS colour picker using hex codes makes these abstract systems immediately relevant.

Learning Activities

Activity 2: Converting Numbers Between Systems

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for pair and group work
- Clear view of board/projector for visual demonstrations of place-value tables

Technology Environment:

- Optional: computers or tablets for digital conversion tools
- Projector/board to display examples and conversion steps

Natural Environment:

- Interactive and supportive environment that encourages collaboration, practice, and healthy competition

Teaching and Learning Materials

- Visual place-value tables (Binary, Denary, Octal, Hexadecimal)
- Worksheets for conversions
- Projector/board
- Pens, markers, and paper for relay race activity
- Chalkboard or whiteboard for live demonstration
- Printed conversion cheat sheet (0–15 across all four systems)
- Flashcards for the relay race activity
- Printed ASCII charts

Suggested Teaching Methods

- **Demonstration:** Show step-by-step conversion methods for each number system
- **Guided Practice:** Pairs practice conversions and check each other's work
- **Independent Practice:** Individual work on denary $\leftrightarrow$ octal and denary $\leftrightarrow$ hexadecimal conversions
- **Collaborative Learning/Relay Race:** Groups solve conversions sequentially to reinforce skills
- **Question and Answer:** Facilitate understanding of practical applications (e.g., Linux file permissions, memory representation)

How to Conduct:

Structure this as a progressive skill-building sequence across three stages. Stage 1 – Binary $\leftrightarrow$ Denary: Introduce the place-value method for binary-to-denary (e.g., $1011_2 = 8+0+2+1 = 11_{10}$) and the repeated-division-by-2 method for denary-to-binary. Demonstrate with a visual "place value table" written on the board (128 | 64 | 32 | 16 | 8 | 4 | 2 | 1). Learners practice in pairs and check each other's answers. Stage 2 – Denary $\leftrightarrow$ Octal: Demonstrate the repeated-division-by-8 method and show how to read remainders from bottom to top. Connect to a Linux file permission context. Learners practice a set of five conversions independently. Stage 3 – Denary $\leftrightarrow$ Hexadecimal: Demonstrate repeated-division-by-16, emphasising the substitution of letters for values 10–15 (A–F). Show how one hex digit represents exactly four binary bits by converting a byte (e.g., $11111111_2 = FF_{16}$). Conclude with a "conversion relay race": each group member solves one conversion step and passes the sheet to the next.

□ **Content Tip:** The repeated-division method works universally for converting from denary to any other base: divide the number by the target base, record the remainder, and repeat with the quotient until it reaches zero. Reading the remainders from bottom to top gives the result in the target base. For hexadecimal, the critical step learners often miss is the substitution: when a remainder is 10, write A; 11 = B; 12 = C; 13 = D; 14 = E; 15 = F. A 'conversion cheat sheet' (0–15 in denary, binary, octal, and hex) is invaluable during the learning phase. Encourage learners to always verify a conversion by converting the result back to the original number – this builds checking habits and reinforces the inverse relationship between the two methods.

Learning Activities

Activity 3: Converting Text Using ASCII

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for pair work and individual engagement
- Clear view of board/projector for step-by-step demonstrations

Technology Environment:

- Optional: computers or tablets to display ASCII tables digitally
- Projector/board to show examples and mystery messages

Natural Environment:

- Interactive environment that encourages hands-on practice, peer sharing, and problem-solving

Teaching and Learning Materials

- Printed ASCII tables
- Worksheets for practice conversions
- Projector/board
- Pens and paper
- Printed ASCII tables for each learner
- Chalkboard or whiteboard for demonstration
- Mystery ASCII message for the decoding challenge

Suggested Teaching Methods

- **Discussion/Hook:** Connect ASCII to storing text in binary
- **Demonstration:** Convert letters to ASCII decimal and binary step-by-step
- **Guided Practice:** Convert example words as a class
- **Individual/Pair Work:** Learners convert letters of their own names and decode each other's results
- **Problem-Solving:** Mystery message decoding activity
- **Question and Answer:** Discuss limitations of ASCII and need for Unicode

▪ How to Conduct:

Begin by connecting to the topic hook: "We know computers store numbers as binary. But how does the computer store your name?" Distribute printed ASCII tables and demonstrate: the capital letter A has the ASCII value 65 (denary), which is 01000001 in binary. Guide learners to convert the word "ZAM" to ASCII decimal values and then to 8-bit binary, showing their working for each letter. Add a personal engagement task: each learner converts the first three letters of their own name to binary and shares with a partner to decode. Extend with a mystery message activity: display a sequence of ASCII decimal values on the board and challenge learners to decode it as a Zambian phrase or motto. Close by asking: "ASCII covers 128 characters. What are its limitations, and why might we need a larger system like Unicode?"

□ **Content Tip:** ASCII (American Standard Code for Information Interchange) assigns a unique number (0–127) to 128 characters: uppercase letters (A=65 to Z=90), lowercase letters (a=97 to z=122), digits (0–9 as 48–57), and common symbols. A key insight to emphasise: uppercase and lowercase versions of the same letter have different ASCII values (A=65, a=97 – a difference of 32). This is why string comparisons in programming are case-sensitive by default. ASCII is the foundation of text encoding in computing, but its 128-character limit is a real constraint. Briefly introduce Unicode (UTF-8) as the modern extension that covers over 143,000 characters including non-Latin scripts like Nyanja, Arabic, Chinese, and emoji – making it the standard for a multilingual world.

Learning Activities

Activity 4: Applying Number System Operations

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for small group work (4 per group) with space to display posters
- Clear view of board/projector for teacher walkthrough and demonstrations

Technology Environment:

- Optional: computers or tablets for verifying conversions digitally
- Projector/board to display sample Gold-level problems

Natural Environment:

- Interactive and competitive environment that encourages collaboration, discussion, and problem-solving

Teaching and Learning Materials

- Problem cards at three difficulty levels (Bronze, Silver, Gold)
- Pens, paper, and markers for showing full working
- Poster sheets for final solutions
- Projector/board for class walkthrough
- Chalkboard or whiteboard for group solutions
- Conversion reference cheat sheets (for Bronze level only)

Suggested Teaching Methods

- **Collaborative Learning:** Groups work together to solve conversion tasks
- **Gamification/Tournament:** Bronze → Silver → Gold problem progression motivates practice
- **Guided Verification:** Teacher or peer checks solutions before advancing
- **Presentation:** Groups post and explain solutions on the board
- **Discussion/Walkthrough:** Class reviews and explains challenging Gold-level problems
- **Peer Learning:** Encourages explanation and reasoning among group members

How to Conduct:

This consolidation activity uses a "conversion tournament" format to review all four systems simultaneously. Prepare a set of problem cards with mixed conversion tasks at three difficulty levels: Bronze (binary $\leftrightarrow$ denary), Silver (octal $\leftrightarrow$ denary and hex $\leftrightarrow$ denary), and Gold (multi-step conversions, e.g., binary $\rightarrow$ hex via denary; or ASCII letter $\rightarrow$ binary). Organise learners into groups of four. Each group starts at the Bronze level; completing all Bronze problems correctly earns them a Silver card. Solutions must be written with full working shown and explained to the teacher or a peer verifier before progressing. Groups post their final solution posters on the board. Conclude with a class walk-through of the most challenging Gold problems, asking different groups to explain each step.

□ **Content Tip:** Multi-step conversions (e.g., converting a word to ASCII, then each ASCII value to hexadecimal, and then grouping the hex digits to find a compact code) are excellent synthesis tasks because they require learners to chain multiple skills together. The most common errors are: (1) forgetting to pad binary numbers to 8 bits (a byte) when representing a character; (2) confusing the ‘from’ and ‘to’ system in a conversion (always write the base subscript, e.g., 1101_2 or $1A_{16}$ to avoid ambiguity); and (3) missing the letter substitution in hexadecimal (writing 10 instead of A). Building the habit of writing subscripts and always verifying by converting back are the two most effective error-prevention strategies.

Expected Standard

Number System Operations applied accordingly

ASSESSMENT

Examples of Formative

Scenario: Your younger cousin in Chipata sees you working on a computer at a community centre and asks, “Bakamba (Elder), I heard that computers only understand 0 and 1. But we are learning about numbers like 10, 25, and 100 in school. How can a computer work with many school numbers if it only knows two digits?”

1. (Binary/Denary) Explain to your cousin how the computer takes the number he sees (like 5) and turns it into a number it understands (like 101). Show him the conversion for the number 5 using a simple place value chart (4s, 2s, 1s).
2. (Hexadecimal) You are helping a friend design a website for a lodge in Livingstone. He wants the exact brown colour of a lechwe antelope. The colour picker shows #8A4117. He’s confused by the letters. Explain to him what this code is and why it’s written this way instead of a long binary number.

Formative Rubric:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Understanding of Number Systems	Clearly explains the base concept and purpose of different systems (binary, octal, hex) in computing.	Identifies the base and digits for binary, denary, octal, and hexadecimal correctly.	Can list digits for binary and denary but confuses octal and hexadecimal.	Does not understand the concept of base or differences between number systems.
Conversion Accuracy (Binary/Denary)	Converts all binary/denary numbers accurately and efficiently, including large numbers.	Converts simple binary/denary numbers with few to no errors.	Converts simple numbers but makes consistent place-value errors.	Unable to perform binary/denary conversions correctly.
Conversion Accuracy (Octal/Hex)	Accurately converts numbers between denary, octal, and hex, including those with letters (A-F).	Converts basic denary numbers to/from octal and hex with occasional errors.	Attempts conversion but makes frequent errors, especially with hex letters.	Unable to perform octal/hex conversions.
Application of ASCII	Converts words to ASCII and then to binary with 100% accuracy	Correctly uses an ASCII table to convert letters to	Makes errors when using the ASCII table or	Cannot use an ASCII table to find character codes.

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
	and explains the process.	decimal numbers.	converting to binary.	
Problem-Solving in Groups	Leads group work, explains methods to peers, and helps correct errors.	Participates actively and contributes to finding solutions.	Participates passively; relies heavily on group for answers.	Does not participate or disrupts group work.

Example of Summative Assessment:

Scenario: A technology club at your school is creating a simple secret code system for sending messages between members. They decide to use ASCII.

Task:

1. Your secret message is "ZAM". Using an ASCII table, write down the decimal number for each letter.
2. Take the decimal number for 'A' (65) and convert it into an 8-bit binary number. Show your working using the place value method (128, 64, 32, 16, 8, 4, 2, 1).
3. Explain to a new club member why using binary, even though it's much longer, is the only way a computer can store the message "ZAM" internally.

SUMMATIVE RUBRIC:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Binary-Denary Conversion	Converts all numbers accurately, showing clear, efficient working.	Converts most numbers accurately with correct method shown.	Converts some numbers correctly but with inconsistent methods or errors.	Cannot convert between binary and denary.
Denary-Octal Conversion	Accurate conversion of all numbers, demonstrating a strong grasp of base-8.	Converts most numbers correctly using repeated division.	Makes several errors in division or ordering of remainders.	Unable to perform denary-octal conversion.
Denary-Hexadecimal Conversion	Accurate conversion of all numbers, including correct substitution of letters.	Converts most numbers correctly, with minor errors.	Makes frequent errors, particularly with the A-F notation.	Unable to perform denary-hex conversion.
ASCII Conversion	Accurately converts a short message to decimal and binary values.	Converts a short message to decimal ASCII values with few errors.	Makes significant errors in converting letters to ASCII values.	Unable to complete the ASCII conversion task.
Explanation of Use	Clearly explains why hex and octal are used in computing contexts (e.g., memory, colour).	Provides a basic reason for using different number systems.	Can name other number systems but cannot explain their purpose.	Cannot explain why anything other than denary is needed.

Summary – Key Points Recap:

- Computers use binary (base-2) because they have two states: on (1) and off (0).
- Different number systems serve different purposes: denary (everyday use), binary (computer processing), octal and hexadecimal (human-friendly representations of binary).
- One hexadecimal digit represents exactly 4 binary bits, making it compact and readable.
- Conversion between number systems requires understanding place values and using systematic methods like repeated division.
- ASCII is the standard for representing text characters as numbers (and ultimately binary).
- All data in a computer – text, numbers, images, sounds – is ultimately represented in binary.

TOPIC 2.3: INTRODUCTION TO PROGRAMMING

Overview

This topic introduces learners to the fundamental concepts of programming through visual, block-based environments. It focuses on how to think logically, sequence instructions, and create simple programs using drag-and-drop coding blocks.

General Competence(s):

Analytical Thinking	Analyses problems and breaks them into smaller steps for programming solutions.
Application Software Skills	Uses block-based programming software to design and execute programs.
Collaboration and Communication	Works in groups to develop, present, and explain programming projects.
Creativity and Innovation	Designs creative solutions and projects using blocks, loops, and events.
Critical Thinking and Problem-Solving	Applies logic to sequence blocks, use conditions, loops, and variables effectively.
Data Structure Skills	Organises and manages program data through variables and sequences.
Digital Literacy	Understands programming concepts and how digital tools implement them.
Programming Skills	Demonstrates ability to construct programs using blocks, events, and logic.
Technical Proficiency	Shows competence in using programming tools and executing programs.

Hook

"Have you ever wondered how video games or animations are created? What if you could tell a computer exactly what to do using colourful blocks instead of typing complex code? Let's explore how you can become a creator, not just a user, of technology!" (Show a short, engaging animation or game created in Scratch to demonstrate what's possible.)

Key Terms / Words / Vocabulary

- **Programming:** The process of writing a set of instructions (code) that tells a computer how to perform a task.
- **Block-Based Programming:** A visual programming method where instructions are represented as interlocking blocks, reducing syntax errors and making it ideal for beginners.
- **Algorithm:** A step-by-step, unambiguous set of instructions to solve a problem.
- **Sequence:** The order in which instructions are executed. Changing the sequence changes the outcome.
- **Loop:** A programming construct that repeats a block of code multiple times.
- **Condition:** A programming construct that makes a decision (if-then-else) – allows different code to execute based on whether a statement is true or false.
- **Variable:** A named storage location that holds a value which can change, like a score in a game.
- **Event:** An action that triggers a specific part of a program to run (e.g., a mouse click, pressing a key).
- **Sprite:** A graphic character or object that can be controlled and moved by code.
- **Script:** A set of programming instructions assembled to control a sprite or perform a task.

Sub-Topic 2.3.1: Block-Based Programming

Introduction: This sub-topic introduces block-based programming as a way of creating programs using visual blocks instead of typing code. Each block represents a command or instruction, and you can drag and join the blocks together to build a program. This type of programming is easy to learn because it reduces errors and helps you understand how programs work step by step. It is commonly used by beginners to create games, animations, and simple applications.

Specific Competence(s) – Learners will:

- 2.3.1.1 Construct programs using block-based software.

Learning Activities

Activity 1: Exploring Different Types of Block-Based Programming Software

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for individual exploration and pair discussion
- Clear view of board/projector for demonstrations and comparisons

Technology Environment:

- Computers or laptops with internet access
- Block-based programming software: Scratch, Blockly, MIT App Inventor, Code.org
- Projector to show interface overview and examples

Natural Environment:

- Supportive and curiosity-driven environment that encourages exploration, questioning, and discovery

Teaching and Learning Materials

- Computers or tablets with Scratch installed or internet access for scratch.mit.edu
- Projector/board for teacher demonstration
- Printed reference guides or screenshots of block-based interfaces
- Note-taking materials for learners' observations
- Printed quick-reference guide to the Scratch interface

Suggested Teaching Methods

- **Exploration/Discovery Learning:** Allow free interaction with the software to spark curiosity
- **Discussion/Q&A:** Learners share discoveries and surprises from exploration
- **Demonstration:** Teacher introduces main areas of the interface (Blocks Palette, Scripting Area, Stage)
- **Comparison:** Highlight differences between platforms and relate to physical LEGO® analogy
- **Guided Introduction:** Explain platform purposes and suitability for different skill levels

▪ How to Conduct:

Begin with a free exploration session: allow learners 10 minutes to open Scratch (online at scratch.mit.edu or the offline editor) and click anything they like without instruction. This removes the anxiety of "doing it wrong" and sparks curiosity. After 10 minutes, ask learners: "What did you discover? What did a block do that surprised you?" Use their discoveries to introduce the three main areas of the Scratch interface: the Blocks Palette (categories of blocks), the Scripting Area (where blocks are assembled), and the Stage (where the program runs). Introduce the concept that different platforms exist for different levels: Scratch and Blockly for beginners, MIT App Inventor for mobile apps, and Code.org for structured tutorials. Compare to physical LEGO®: blocks only connect in logical ways, preventing the equivalent of "syntax errors" in text-based languages.

□ **Content Tip:** Block-based programming environments are deliberately designed so that blocks only snap together in ways that make logical sense – an Event block ("When Green Flag Clicked") must come first, a Motion block ("Move 10 Steps") goes in the middle, and so on. This structural constraint prevents the most common beginner frustration in text-based programming: syntax errors. Make this comparison explicit: "In Python, if you miss a colon or indent wrongly, the code breaks. In Scratch, the blocks simply won't connect if they don't belong together." This helps learners understand why block-based programming is a powerful learning bridge to text-based languages – it teaches logic and structure without the syntax burden.

Learning Activities

Activity 2: Describing the Key Features of Block-Based Programming

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for pair work with space for individual hands-on practice
- Clear view of board/projector for live script demonstration

Technology Environment:

- Computers or laptops with Scratch installed or accessible online
- Projector to show interface and example scripts

Natural Environment:

- Supportive, focused environment encouraging observation, discussion, and experimentation

Teaching and Learning Materials

- Interface map worksheets with numbered blank labels
- Computers/laptops with Scratch open
- Projector/board for live demonstration
- Pens, markers, and paper for notes
- Printed interface map worksheet with numbered labels

Suggested Teaching Methods

- **Pair Work:** Identify and label key interface areas collaboratively
- **Demonstration:** Teacher shows a simple three-block script and predicts outcome
- **Guided Practice:** Learners recreate the script independently
- **Question and Answer:** Discuss the function of each block and importance of event blocks
- **Observation & Prediction:** Learners predict script behaviour before execution

How to Conduct:

Provide learners with a structured "interface map" worksheet showing a screenshot of the Scratch interface with numbered blank labels. Learners work in pairs to identify and name each area: Stage, Sprite List, Blocks Palette, Scripting Area, and the green flag/red stop controls. After labelling, introduce the concept of a script by demonstrating a simple three-block script live: "When Green Flag Clicked" → "Move 10 Steps" → "Play Sound [Meow]". Ask learners to predict what will happen before running it. Learners then recreate this exact script independently. Debrief: "What did each block do? What would happen if we removed the event block at the top? Would the script run automatically?" – establishing that an event block is required to start a script.

□ **Content Tip:** Emphasise the distinction between the four main block categories as a framework for thinking about code structure. Event blocks (yellow, hat-shaped) answer the question: "When does this code run?" Motion and Sound blocks (blue and pink) answer: "What does the sprite do?" Control blocks (orange) answer: "How many times? Under what conditions?" Sensing blocks (light blue) answer: "What does the sprite know about its environment?" This categorical thinking maps directly onto real programming: event handlers, actions, control flow, and input/output. Framing it this way from the beginning builds the mental model that transfers to text-based languages.

Learning Activities

Activity 3: Identifying Basic Programming Constructs

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for pair work and easy movement for card game
- Space to display and discuss learners' annotations

Technology Environment:

- Computers/laptops with Scratch
- Projector/board to show example projects and Scratch interface

Natural Environment:

- Interactive and supportive environment encouraging collaboration, discussion, and hands-on learning

Teaching and Learning Materials

- Printed matching cards (concept cards and block screenshot cards)
- Computers/laptops with Scratch projects for the live project exploration
- Printed screenshots of an existing Scratch project for annotation
- Projector/board
- Pens, markers, and paper for notes

Suggested Teaching Methods

- **Pair Work/Matching Game:** Match programming concepts with Scratch blocks
- **Guided Practice:** Identify constructs in existing Scratch projects
- **Annotation & Presentation:** Label blocks on printed screenshots and explain usage
- **Discussion/Q&A:** Share findings and clarify understanding of constructs
- **Hands-on Learning:** Apply concepts in real projects to reinforce understanding

▪ How to Conduct:

Create a physical matching game using printed cards. Set A contains concept cards: "Loop", "Condition", "Variable", "Event". Set B contains screenshot cards showing a corresponding Scratch block (e.g., "Repeat 10" block, "If <> Then" block, "Set [Score] to 0" block, "When Green Flag Clicked" block). Learners work in pairs to match each concept to its block, then explain in their own words what each block does and when they would use it. After the card game, open Scratch and navigate to an existing project (the Scratch community has thousands of examples). Learners identify at least one example of each construct in the real project's code, annotating a printed screenshot with arrows and labels. Consolidate by having each pair share one construct they found and explain it to the class.

□ **Content Tip:** These four constructs are the universal building blocks of all programming languages, not just Scratch. A loop (Repeat, Forever) in Scratch becomes a for or while loop in Python. A condition (If/Then/Else) becomes an if statement. A variable (Set Score to 0) becomes a variable declaration. An event (When Green Flag Clicked) becomes a main() function or an event listener in JavaScript. Making this cross-language connection early helps learners see Scratch as a stepping stone, not a destination, and motivates the deeper study of programming constructs. A useful classroom display: a "Scratch vs. Python" poster showing equivalent code structures side by side.

Learning Activities

Activity 4: Constructing Simple Programs by Sequencing Blocks

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for individual hands-on practice with teacher support
- Clear view of board/projector for live coding demonstration

Technology Environment:

- Computers/laptops with Scratch installed or accessible online
- Projector to display live coding and step-by-step instructions

Natural Environment:

- Supportive and focused environment encouraging guided learning, experimentation, and independent problem-solving

Teaching and Learning Materials

- Computers/laptops or tablets with Scratch open
- Projector/board for live demonstration (showing teacher's live coding)
- Printed step-by-step guides or screenshots (optional)
- Pens and paper for planning sprite movement or logic
- Step-by-step guided worksheet for the MVP project

Suggested Teaching Methods

- **Demonstration:** Teacher demonstrates coding a minimal program with think-aloud strategy
- **Guided practice** where learners follow along and teacher monitors progress
- **Independent Practice:** Independent extension of the project with additional movement, boundaries, and sprite actions
- **Observation & Feedback:** Teacher checks learners' work and provides guidance
- **Hands-on Practice:** Encourages computational thinking and sequencing skills

▪ How to Conduct:

Use the "I Do, We Do, You Do" instructional structure. **I Do:** Teacher live-codes a Minimal Viable Product (MVP) – a sprite that moves right when the right arrow key is pressed and left when the left arrow key is pressed. Think aloud throughout: "First I need an event block to detect the key press. Then I need a motion block. Let me test it after each block I add." **We Do:** Learners follow along step-by-step from the board to recreate the same project, with the teacher checking progress at each step. **You Do:** Learners independently extend the project by adding: (a) up and down movement, (b) a boundary so the sprite cannot go off-screen (using an If on edge, bounce block or a conditional), and (c) a costume change when the sprite moves. The independence challenge – extending beyond the guided steps – is where genuine computational thinking develops.

□ **Content Tip:** "Start simple and add features" is the core discipline of software development. Build a Minimal Viable Product (MVP) first – the smallest version that does something useful – then add complexity incrementally. Test after every addition. This practice directly parallels professional software development methodology and prevents "spaghetti code" (an overwhelming, untestable tangle of blocks). A practical debugging habit to model from the first lesson: if the program is not working, do not delete everything and start over. Instead, check the sequence (are the blocks in the right order?), check the connections (are all blocks attached?), and test the simplest possible version first, then add back components one by one until the error reappears.

Learning Activities

Activity 5: Using Loops, Conditions, Variables, and Events in a Project

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for individual and pair work with space for planning and peer-sharing
- Clear view of board/projector for live demonstrations and debugging examples

Technology Environment:

- Computers/laptops with Scratch
- Projector/board to display examples and debugging demonstrations

Natural Environment:

- Interactive and supportive environment encouraging planning, experimentation, problem-solving, and peer collaboration

Teaching and Learning Materials

- Computers/laptops or tablets with Scratch
- Project planning sheets templates for project design (sprites, variables, events, conditions)
- Printed screenshots of example projects
- Pens, markers, and paper for notes and planning
- Projector for teacher demonstration and peer showcase

Suggested Teaching Methods

- **Demonstration:** Show how to integrate loops, conditions, variables, and events into a project
- **Guided Practice:** Learners follow step-by-step instructions to build their interactive story or game
- **Planning and Design:** Complete planning sheets before coding
- **Independent Practice:** Build and debug projects individually or in pairs
- **Peer Learning/Showcase:** Share completed projects and discuss debugging strategies
- **Problem-Solving:** Identify and fix intentional coding errors to reinforce understanding

▪ How to Conduct:

Guide learners to build a simple interactive story or game that explicitly requires all four constructs. Suggested project: "Zambia Nature Quiz" – a sprite asks a multiple-choice question about a Zambian animal; the learner presses 1, 2, or 3; the sprite responds with correct/incorrect feedback; a Score variable increases by 1 for each correct answer; and a Forever loop keeps the quiz running. Before coding, learners complete a planning sheet listing: sprites needed, variables needed, events that trigger actions, and conditions that control branching. After building, introduce the concept of debugging by intentionally placing the Score variable increment block in the wrong place (outside the If block) and challenging learners to identify and fix it. Conclude with a mini peer-showcase.

□ **Content Tip:** Planning before coding is one of the most important habits to instil. A project plan does not need to be formal – even a simple list of "sprites I need, variables I need, and events that happen" prevents learners from starting with no direction and becoming overwhelmed. This planning mindset maps directly onto professional software development practices (user stories, pseudocode, wireframes). For debugging practice, intentional bugs are more pedagogically effective than accidental ones: the teacher controls the type and complexity of the bug, and it ensures all learners encounter the same problem simultaneously. Common bugs to introduce: a variable not being reset when the game starts (add "Set Score to 0" under the Green Flag event); a condition check with the wrong operator (greater than vs. less than); or a sound block placed outside a loop so it only plays once.

Learning Activities

Activity 6: Presenting and Sharing Projects

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for movement between workstations (Studio Walk format)
- Space for learners to circulate safely in small groups
- Clear view of board/projector for final volunteer presentations

Technology Environment:

- Computers/laptops with Scratch projects open
- Projector/board for final class presentations

Natural Environment:

- Collaborative and reflective environment that encourages peer feedback, discussion, and iterative improvement

Teaching and Learning Materials

- Computers with Scratch (all learner projects open)
- Computers/laptops with projects ready to share
- Sticky notes and pens for feedback
- Projector/board for the three-learner volunteer showcase presentations
- Pens, markers, and paper for notes
- Optional charts or posters for visual feedback reminders

Suggested Teaching Methods

- **Peer Review/Studio Walk:** Rotate in small groups to interact with each project, read code, and leave constructive feedback
- **Guided Reflection:** Learners review feedback and implement actionable suggestions
- **Presentation:** Selected learners explain their project, highlighting loops and bug fixes
- **Collaborative Learning:** Encourage discussion on design choices and coding solutions
- **Hands-on Iteration:** Make improvements based on peer feedback to reinforce learning

▪ How to Conduct:

Structure the sharing session as a "Studio Walk" inspired by an art gallery. Each learner's project is open on their computer screen. Learners rotate around the room in groups of three, spending three minutes at each computer: one minute playing the project, one minute reading the code (looking at the scripts), and one minute leaving written feedback on a sticky note (one thing they liked, one question they have, one suggestion). After the rotation, each learner reads the feedback on their own project and has five minutes to implement the most actionable suggestion. The lesson closes with three volunteers presenting their project to the whole class, explaining: (a) what their program does, (b) where they used a loop, and (c) one bug they fixed during development.

□ **Content Tip:** The "remix culture" in Scratch – the ability to look inside any public project, copy it, and modify it – is a powerful and legitimate learning tool. Examining expert code to understand how a specific effect was achieved (e.g., how did they make the sprite bounce realistically? How did they store the high score?) is exactly how professional developers learn. Teach learners the vocabulary of constructive code review using structured sentence starters: "I really liked how you used [construct] to achieve [effect]...", "Have you considered using a variable to [improvement]...", "I noticed the score doesn't reset when the game restarts – have you thought about how to fix that?" This language scaffold ensures feedback is specific, technical, and respectful.

Expected Standard

Block-based programs constructed correctly.

ASSESSMENT

Examples of Formative

Scenario: In your Scratch class, a learner named Chileshe has created a simple animation of a goat (sprite). He wants the goat to walk 10 steps, then bleat, then walk another 10 steps. He has put the blocks in this order: play sound "Meee" → move 10 steps → move 10 steps.

1. (Debugging) When Chileshe runs his program, the goat bleats first, then moves 20 steps all at once. What is wrong with his sequence? Explain how he should reorder the blocks to make it work as he intended (walk, bleat, walk).
2. (Identifying Constructs) Chileshe now wants the goat to keep walking and bleating forever. What type of programming construct (loop, condition, event) does he need to add, and where should it go around his blocks?

Formative Rubric:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Identifying Constructs	Can explain and identify loops, conditions, variables, and events in existing and new code.	Correctly identifies loops, conditions, variables, and events in a given script.	Can identify some constructs (e.g., loops) but confuses others (e.g., event vs. condition).	Cannot identify basic programming constructs.
Sequencing & Logic	Creates complex, multi-step sequences that function efficiently and logically.	Creates a correct sequence of blocks to achieve a simple goal.	Sequence has logical flaws or out-of-order blocks that prevent the program from working.	Blocks are placed randomly with no logical sequence.
Debugging	Independently identifies and fixes errors in code, explaining the cause of the bug.	With teacher prompting, can find and fix errors in their own or provided code.	Attempts to fix errors but often creates new ones or cannot find the source.	Does not attempt to fix errors; restarts from scratch or gives up.
Use of Constructs	Independently and appropriately uses loops, conditions, and variables to enhance a project.	Uses loops and conditions as directed to control a program's flow.	Attempts to use loops/conditions but implements them incorrectly.	Only uses sequential blocks; no evidence of loops or conditions.
Collaboration & Peer Review	Provides constructive, specific feedback to peers and incorporates feedback to improve own work.	Listens to peers and provides basic feedback.	Provides off-topic or unhelpful feedback.	Does not participate in peer review.

Example of Summative Assessment:

Scenario: You are tasked with creating a short, interactive “Independence Day Dance Party” animation in Scratch for a school assembly in Solwezi.

Task:

1. The program must start when the green flag is clicked (an event).
2. A sprite should change costumes to give the appearance of dancing. Use a loop to make the costume changes continue.
3. When the sprite is clicked (another event), a drum sound should play.
4. Include a variable called “Energy” that starts at 100 and decreases by 1 every time the sprite dances a full cycle.
5. After presenting your project, explain how you used a loop and an event in your code.

SUMMATIVE RUBRIC:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Project Functionality	Program runs without errors and includes multiple sprites/interactions. All features work as intended.	Program runs with few minor bugs and achieves the core goal.	Program runs but has significant bugs that prevent full functionality.	Program does not run or does not meet the basic goal.
Use of Constructs	Skilfully combines sequences, loops, conditions, and variables creatively to solve problems.	Correctly uses sequences, loops, and at least one condition or variable.	Uses sequences and attempts to use a loop or condition, but with errors.	Uses only sequential blocks; no loops, conditions, or variables.
Design & Creativity	Project is highly original, engaging, and shows attention to user experience.	Project shows creativity and is visually organised.	Project is a direct copy of a tutorial with minimal personalisation.	Project is incomplete or shows no effort in design.
Modification Task	Successfully modifies code to add a new, complex feature.	Successfully modifies code to add a simple, specified feature.	Attempts to modify code but introduces errors or cannot complete the task.	Cannot modify the given code.
Code Explanation	Clearly and confidently explains how their code works, including purpose of key blocks.	Can explain the basic purpose of their code when asked.	Can identify parts of their code but struggles to explain how they work together.	Cannot explain their code.

Summary – Key Points Recap:

- Block-based programming makes coding accessible and fun by removing syntax errors.
- Basic programming constructs include sequences, loops (repeat), conditions (if-else), variables, and events.
- Planning and testing are essential steps in creating a program.
- Coding develops logical thinking, creativity, and problem-solving skills.

TOPIC 2.4: NETWORKING

Overview

Computer networking is the process of connecting multiple computing devices to enable the sharing of data, software, and hardware resources. This topic focuses on network topologies – the design and layout of network connections – and how they determine how information flows across a Local Area Network (LAN).

General Competence(s):

Analytical Thinking	Analyses different network topologies and evaluates their suitability for specific scenarios.
Collaboration and Communication	Works in teams to design, build, and configure networks while clearly explaining decisions.
Creativity and Innovation	Designs original and efficient network layouts to solve connectivity challenges.
Critical Thinking and Problem-Solving	Identifies and resolves network design or configuration problems logically.
Technical Proficiency	Demonstrates hands-on skills in building, configuring, and troubleshooting networks.
Digital Literacy	Understands networking concepts, devices, protocols, and their practical applications.

Hook

"How can a single printer in the school office be used by every teacher without anyone leaving their classroom?" (Follow up: "What happens if the cable to that printer breaks? Can anyone print then?")

Key Terms

Topology: The physical or logical layout of devices and connections in a network – the “map” of the network.

Ethernet: The most common technology for connecting devices in a wired Local Area Network (LAN).

UTP / STP (Unshielded/Shielded Twisted Pair): Copper cabling used in Ethernet networks. STP has extra shielding to reduce interference in noisy environments.

IP Address: A unique numerical label assigned to each device on a network for communication and identification.

LAN (Local Area Network): A network that connects devices within a limited area, such as a school, home, or office building.

Router: A device that connects different networks together and directs traffic between them.

Switch: A device that connects multiple devices on the same network and forwards data only to its intended destination.

Sub-Topic 2.4.1: Network Topologies

Introduction: This sub-topic introduces network topologies as the different ways computers and devices are arranged and connected in a network. These arrangements determine how data moves from one device to another. Common types of network topologies include bus, star, ring, mesh, and tree. Understanding network topologies helps you know how networks are organised and how they work efficiently.

Specific Competence(s) – Learners will:

- 2.4.1.1 Develop and configure network topologies.

Learning Activities

Activity 1: Discussing Different Types of Network Topologies

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for movement and group interaction to model physical networks
- Space to safely form Star, Bus, Ring, and Mesh topologies with string and learners

Technology Environment:

- Projector or screen to display diagrams of network topologies after the physical activity
- Computers for reference to real network layouts

Natural Environment:

- Supportive, interactive environment encouraging hands-on exploration, observation, and discussion

Teaching and Learning Materials

- Strings or ropes for the kinaesthetic human network activity to model network connections
- Labels or badges for roles (switch, computer, server)
- Projector/board for diagrams
- Printed network topology diagrams (Star, Bus, Ring, Mesh)
- Pens, markers, and paper for notes and Worksheets or activity sheets for reflection
- Network simulation software (e.g., Cisco Packet Tracer) for follow-up

Suggested Teaching Methods

- **Kinesthetic Learning:** Physically model each network topology using learners and strings
- **Demonstration:** Show diagrams and explain the structure after the hands-on activity
- **Discussion/Questioning:** Compare topologies and discuss advantages, limitations, and real-life applications
- **Reflection:** Ask learners to identify which topology matches their school's lab network
- **Collaborative Learning:** Encourage teamwork while forming topologies and passing messages

▪ How to Conduct:

Use a kinaesthetic "human network" activity before showing any diagrams. Arrange learners physically to model each topology using lengths of string as cables and learners as computers. For a Star topology: one learner stands in the centre (the switch) and holds a piece of string to each of the other learners; all communication goes through the centre. For a Bus topology: learners stand in a line and pass a message (a piece of paper) along a single string – show that if the string breaks anywhere, nobody further down can communicate. For a Ring topology: learners form a circle, each holding both ends of adjacent strings. For a Mesh topology: every learner holds strings to multiple others. After each physical model, show the corresponding network diagram and identify the labels. Conclude by discussing: "Which topology best describes your school's computer lab network?"

□ **Content Tip:** Each topology involves a fundamental trade-off between cost, reliability, and ease of management. Star topology: all devices connect to a central switch. If one cable fails, only that device is affected – making it easy to troubleshoot. However, if the switch fails, the entire network goes down. This is the most common topology in schools and offices. Bus topology: all devices share a single backbone cable. It is cheap and simple but highly vulnerable – a break anywhere on the backbone disconnects all devices. Ring topology: data travels in one direction around a loop. A break anywhere disrupts the entire network unless a dual-ring is used. Mesh topology: every device connects to every other device. It is extremely resilient (multiple alternative paths for data) but very expensive due to the number of cables required. Mesh is used in critical infrastructure like military networks and internet backbone connections.

Learning Activities

Activity 2: Analysing Roles of Network Topology

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for small group discussions and presentations
- Space for group debate and peer feedback

Technology Environment:

- Projector/board to display scenario instructions or sample diagrams
- Computers or tablets for research on hardware requirements and topology types

Natural Environment:

- Collaborative, reflective environment encouraging analysis, critical thinking, and structured debate

Teaching and Learning Materials

- Scenario cards describing different network environments
- Worksheets or report templates for recommendations
- Projector/board for visual prompts and diagram examples
- Pens, markers, and paper for calculations and notes
- Optional reference charts for hardware requirements and topology features
- Topology recommendation report template
- Cost-benefit analysis comparison table (printed)

Suggested Teaching Methods

- **Collaborative Learning:** Groups analyse scenarios and complete recommendation reports
- **Problem-Based Learning:** Apply knowledge of topologies to real-world situations
- **Presentation & Debate:** Groups present their solution and discuss its advantages and risks
- **Guided Discussion:** Teacher facilitates comparison of different topologies and decision-making criteria
- **Critical Thinking:** Evaluate pros, cons, and risks to justify topology choices

▪ How to Conduct:

Present learners with three scenario cards describing real-world environments: (A) a 30-computer school computer lab in a single room with a full-time technician; (B) a small rural clinic with four computers in separate rooms and a limited budget; (C) a large bank with 200 workstations across three floors that cannot afford any network downtime. Groups of three analyse their scenario card and complete a topology recommendation report: (1) Which topology do you recommend? (2) What hardware would you need (how many switches, how much cable)? (3) What are the main risks of this topology for this scenario? (4) Why is it better than the alternatives? Groups present their recommendation to the class, and the class votes on whether they agree with the choice, generating a structured debate.

□ **Content Tip:** Topology selection is fundamentally an engineering decision that balances four factors: cost (how much cable and how many network devices?), reliability (what happens when a single component fails?), scalability (how easily can new devices be added?), and performance (does the topology create bottlenecks?). Provide learners with a simple cost-benefit analysis table to fill in for each topology across these four dimensions. The key insight is that no topology is universally "best" – the correct answer always depends on the specific constraints and requirements of the environment. This is a powerful lesson in applied problem-solving that extends beyond networking to engineering and design thinking.

Learning Activities

Activity 3: Creating Network Cables (Ethernet, UTP, STP)

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom or lab arranged with individual workstations for hands-on cable creation
- Clear space for teacher demonstration and safe handling of tools

Technology Environment:

- Projector or screen to show video demonstrations
- Cable testers and computers for testing network cables

Natural Environment:

- Safe, structured, and supervised environment promoting practical skill development and attention to safety

Teaching and Learning Materials

- RJ45 connectors, UTP/STP cables, scissors, cable strippers, crimping tools
- Printed T568B wiring diagrams and colour-order charts
- Cable testers for verification
- Projector/board for video demonstration and step-by-step instructions
- Activity sheets to record cable test results and errors
- Networking toolkits per group: UTP/STP cable, RJ45 connectors, crimping tools, cable strippers
- Safety guidelines handout

Suggested Teaching Methods

- **Demonstration:** Teacher shows cable-crimping steps and safety precautions
- **Guided Practice:** Learners follow step-by-step instructions to make a cable
- **Collaborative Learning:** Pair work to memorise T568B colour order and check each other's work
- **Hands-On Practice:** Each learner creates, tests, and corrects their own cable
- **Problem-Solving:** Identify errors in failed cables and correct them

▪ How to Conduct:

Before allowing learners to handle tools, show a two-minute video demonstrating the crimping process and the T568B wiring standard. Discuss safety: RJ45 connectors have sharp edges; scissors and cable strippers must be handled carefully. Distribute a printed T568B colour-order diagram (White/Orange, Orange, White/Green, Blue, White/Blue, Green, White/Brown, Brown) and ask learners to memorise the order by colour in pairs. Then demonstrate live: (1) Strip 2.5 cm of the outer jacket without cutting the inner wires; (2) Untwist and straighten the wires; (3) Arrange in T568B order; (4) Trim to 1.5 cm; (5) Insert into the RJ45 connector, ensuring all eight wires reach the end; (6) Crimp firmly. Learners then make their own cable. Test every cable with a cable tester and record results (pass/fail for each of the 8 pins). Any failed cable is disassembled, the error identified, and the cable remade.

□ **Content Tip:** Three practical tips for successful cable crimping that save time and materials: First, after untwisting the wires, run your thumbnail along each wire to straighten it – this makes ordering them much easier. Second, after arranging the wires in the correct order, hold them tightly between your thumb and forefinger and make a single clean cut to ensure all eight ends are level before inserting into the RJ45 connector – uneven ends are the most common cause of cable failures. Third, before crimping, look through the transparent RJ45 connector housing and verify that all eight wires are visible at the front edge – if any wire is short, pull them out and adjust. A cable tester is non-negotiable: always test before installing a cable in a network to avoid damaging connected equipment.

Learning Activities

Activity 4: Designing Network Topologies

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for small group collaboration, with tables allowing groups to spread out printed stamps and sketch diagrams
- Space for peer review and presentation of designs

Technology Environment:

- Projector/board for example diagrams and symbols
- Computers for optional research on IP addressing and topology planning

Natural Environment:

- Supportive, reflective environment encouraging collaboration, creativity, and problem-solving

Teaching and Learning Materials

- Printed network component icon stamps (computers, switches, router, printer, server)
- Paper and markers for sketching diagrams
- Worksheets with scenario briefs and calculation templates
- Projector/board to display standard network symbols
- Activity sheets for peer review and annotation
- Large paper or whiteboards for diagram drawing
- Standard network diagram symbol reference sheet
- Access to draw.io or Google Drawings if computers are available

Suggested Teaching Methods

- **Collaborative Learning:** Groups design networks together using printed stamps before finalising diagrams
- **Guided Practice:** Teacher provides hints on topology selection, switch count, and IP allocation
- **Peer Review:** Another group evaluates the diagram for clarity, correctness, and completeness
- **Hands-On Learning:** Physically arranging components before drawing reinforces spatial understanding
- **Problem-Solving:** Decide on topologies and component placement to meet scenario requirements

▪ How to Conduct:

Provide each group with a scenario brief and a set of printed network component "stamps" (icons of computers, switches, routers, printers, and servers) that they can arrange physically on paper before drawing their final diagram. The scenario: "Design a network for a new secondary school with 2 computer labs (15 computers each), a library (5 computers), a staff room (8 computers), and a principal's office (2 computers). The school has a single internet connection." Groups must: (1) decide on the topology for each section and for connecting sections together; (2) calculate the minimum number of switches needed; (3) draw a labelled network diagram using standard symbols; (4) annotate with IP address ranges for each section (e.g., Lab 1 uses 192.168.1.x). Diagrams are reviewed by another group who must answer: "Can you identify what every component is and why it is placed there?"

□ **Content Tip:** Professional network diagrams use standardised symbols (Cisco icon set is the industry standard) and always include: device labels, IP address schemes, cable types, and connection ports. For this level, the critical conceptual point is the hierarchical network structure used in schools and offices: multiple switches (access layer) connect workstations to a central, higher-capacity switch or router (distribution/core layer) which connects to the internet. This hierarchical model is more scalable and easier to manage than connecting everything to a single switch. When assigning IP addresses, explain the concept of subnets: grouping all devices in Lab 1 under 192.168.1.x and Lab 2 under 192.168.2.x makes the network easier to manage and troubleshoot.

Learning Activities

Activity 5: Creating and Configuring Network Topologies (Simulation)

Learning Environment Setup

Artificial (Physical – Classroom):

- Computer lab with individual workstations for each learner or pair

Technology Environment:

- Computers with Cisco Packet Tracer installed
- Projector/board to demonstrate placement, configuration, and troubleshooting steps
- Internet optional for reference and troubleshooting guides

Natural Environment:

- Structured, supportive environment promoting hands-on experimentation, troubleshooting, and logical thinking

Teaching and Learning Materials

- Computers with Cisco Packet Tracer installed (free from Cisco NetAcad)
- Projector/board for live demonstration, Printed diagrams of the designed network from Activity 4
- Worksheets or activity sheets to record IP addresses, connectivity tests, and troubleshooting steps
- Sample scenario cards to guide network configuration, Step-by-step Packet Tracer configuration guide, Pre-configured sample network files for reference

Suggested Teaching Methods

- **Demonstration:** Teacher shows step-by-step device placement, cabling, and IP configuration
- **Guided Practice:** Learners replicate the network simulation on their computers with teacher support
- **Hands-On Learning:** Learners assign IPs, connect devices, and test connectivity using ping
- **Problem-Solving & Troubleshooting:** Identify and fix connection issues using logical steps
- **Collaborative Learning:** Pair work to discuss configuration choices and troubleshoot together

▪ How to Conduct:

Using Cisco Packet Tracer (free educational software), learners build a digital version of the network they designed in Activity 4. Guide them through placing devices from the Device Panel, connecting them with the correct cable type (straight-through for switch-to-computer connections, crossover for switch-to-switch connections – or use the "Automatically Choose Connection Type" option for beginners), and assigning IP addresses. For each PC: open the Desktop tab, select IP Configuration, and enter an IP address and subnet mask. Test connectivity by opening the Command Prompt on one PC and using the ping command (e.g., ping 192.168.1.2) to verify that packets reach another device. If ping fails, guide learners to troubleshoot by checking: (1) Is the cable connected? (2) Are the IP addresses in the same subnet? (3) Is any device powered off?

□ **Content Tip:** Packet Tracer's Simulation Mode is an outstanding teaching tool for visualising abstract networking concepts. In Simulation Mode, learners can send a single packet and watch it travel through the network step by step, seeing exactly which device it passes through and how the switch's MAC address table directs it to the correct port. This makes the otherwise invisible process of data switching tangible and observable. Key concepts to reinforce during configuration: each device needs a unique IP address in the same subnet to communicate. The subnet mask (typically 255.255.255.0 for a simple /24 network) defines the boundary of the network – devices with different subnet addresses cannot communicate without a router. The ping command is the universal first diagnostic tool in networking: if ping succeeds, the basic network path is working.

Learning Activities

Activity 6: Building Physical Topologies and Configuring a LAN

Learning Environment Setup

Artificial (Physical – Classroom):

- Computer lab arranged with group workstations for hands-on LAN building
- Space for moving around safely with cables and networking hardware

Technology Environment:

- Computers with network interface cards
- Switches, routers, RJ45 cables (created in Activity 3)
- Projector/board for demonstration of IP settings and troubleshooting

Natural Environment:

- Safe, collaborative environment fostering practical application, observation, and problem-solving

Teaching and Learning Materials

- Ethernet cables (UTP/STP) previously created by learners
- Network switches (one per group of 3–4 learners) Switches and routers for LAN connections
- Computers for workstations
- Projector/board for live demonstration of IP configuration
- Worksheets to record IP addresses, ping test results, and troubleshooting notes
- Printed guides for step-by-step LAN setup
- Workstations or laptops / Optional: a router for the inter-LAN extension challenge

Suggested Teaching Methods

- **Demonstration:** Teacher shows IP configuration, LAN setup, and ping testing
- **Hands-On Learning:** Groups physically connect computers to switches and assign IPs
- **Guided Practice:** Teacher supports learners in troubleshooting connectivity issues
- **Collaborative Learning:** Groups work together to connect LANs through a router
- **Problem-Solving:** Diagnose failures such as missing default gateways and fix issues

▪ How to Conduct:

This is the culminating practical activity, combining cable creation (Activity 3), topology design (Activity 4), and IP configuration (Activity 5) into a physical build. Each group uses the cables they created in Activity 3 to connect three workstations to a switch, establishing a physical Star LAN. With teacher guidance, each group assigns IP addresses manually to each computer (right-click Network icon > Properties > TCP/IP Settings) ensuring they are on the same subnet (e.g., 192.168.10.1, 192.168.10.2, 192.168.10.3, all with subnet mask 255.255.255.0). Test connectivity using the ping command from each computer. Once the basic LAN works, extend the challenge: connect two groups' LANs together through a router and configure the router's IP addresses as gateways. Learners observe that without a default gateway configured, inter-LAN communication fails.

□ **Content Tip:** A physical build session is where conceptual understanding becomes visceral, real competence. When a learner's ping command returns "Reply from 192.168.10.2" for the first time, the abstract concept of network addressing becomes concrete. A critical classroom management tip: label all cables with their destination (e.g., "PC1 to Port 1") before starting the build. Unlabelled cables in a multi-group session become impossible to troubleshoot. The most common physical errors are: (1) cable connected to the wrong port (switch ports versus router WAN ports look similar); (2) cable not fully seated (check the RJ45 clip is clicked in); (3) IP address typo (192.168.10.1 vs 192.168.10.11). These are exactly the errors a professional network technician encounters daily – learning to diagnose and fix them systematically is a highly transferable real-world skill.

Expected Standard

Network topologies are developed and configured correctly.

ASSESSMENT

Examples of Formative

Scenario: A small clinic in your area has four computers connected to a single switch in the reception area. If the cable to one consultation room is unplugged, the other doctors and the receptionist can still work.

1. (Topology Identification) What network topology is the clinic using? Explain why the failure of one cable doesn't affect the others.
2. (Configuration & Testing) The clinic buys a new computer for a fourth consultation room. After connecting it to the switch, the new computer cannot access the internet or see the other computers. What is one specific thing you could check or test to diagnose the problem?

Formative Rubric:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Topology Identification	Can explain advantages/disadvantages of each topology and recommend for real-world scenarios.	Correctly identifies Star, Bus, Ring, and Mesh topologies from diagrams.	Identifies some topologies but confuses others (e.g., Ring vs. Mesh).	Cannot identify different network topologies.
Cable Creation	Independently creates a correctly wired, fully functional Ethernet cable that passes the tester.	Creates a functional Ethernet cable with correct wiring after guidance.	Creates a cable but with wiring errors; cable tester shows faults.	Unable to create a cable; shows no understanding of wiring standards.
Physical Topology Setup	Successfully builds a Star topology and can extend it with additional switches, explaining each device's role.	Connects workstations to a switch correctly to form a basic LAN.	Attempts to connect devices but makes errors (e.g., wrong ports, no power).	Cannot physically connect devices to form a network.
Configuration & Testing	Configures IP addresses and successfully tests connectivity using ping; troubleshoots issues independently.	With guidance, assigns IP addresses and tests connectivity between devices.	Attempts configuration but makes addressing errors; cannot successfully ping.	Does not understand IP addressing or how to test connectivity.
Terminology Knowledge	Accurately uses terms like switch, router, IP, LAN, topology in explanations.	Can define most key terms correctly in a quiz.	Confuses some terms (e.g., switch vs. router).	Cannot define basic networking terms.

Example of Summative Assessment:

Scenario: A new Internet café is opening in Kasama with 8 computers for customers. The owner, Mr. Tembo, needs a reliable network. If one computer has a problem, it should not disrupt the others.

Task:

1. Cable Creation: Create one working Ethernet cable using UTP cable and RJ45 connectors. Test with a cable tester.
2. Topology Setup: Using a switch and the cables you made, physically connect three computers to demonstrate the café's network. What is this topology called?
3. Network Plan: Draw a simple plan showing how you would connect 15 computers. Label the main hardware you would need. Explain in one sentence why the Star topology is a good fit for a cybercafé.

SUMMATIVE RUBRIC:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Cable Creation (2 cables)	Both cables are correctly wired, fully functional, and neatly assembled.	Both cables are functional after testing; minor issues in neatness.	One cable works; the other has wiring errors or both are fixed with help.	Neither cable works; significant wiring errors.
Star Topology Setup	Devices are correctly connected and all can communicate; demonstrates troubleshooting ability.	Three devices are correctly connected to a switch and can communicate (ping).	Devices are connected but some are not communicating due to connection errors.	Unable to establish a working network connection between devices.
Network Plan Design	Plan is detailed, includes topology choice, device list, IP scheme, and cost estimate; well-justified.	Plan includes topology choice, device list, and basic layout for the scenario.	Plan is missing key elements or chosen topology is unsuitable for the scenario.	Plan is incomplete or does not address the scenario.
Justification of Choice	Provides thorough, logical reasoning for topology selection based on cost, reliability, and scalability.	Provides a basic reason for choosing a specific topology.	Justification is weak or not aligned with the scenario.	No justification provided.

Summary – Key Points Recap:

- Network topology defines how devices are connected and how data travels.
- Star, Bus, Ring, and Mesh are common topologies, each with unique strengths and weaknesses.
- Hands-on skills like cable creation and switch configuration are essential for building real networks.
- Planning, testing, and ethical considerations are key to successful network implementation.

TERM TWO

TOPIC 2.5: CYBER SECURITY

Overview

Cyber security focuses on keeping digital devices and information safe from threats such as viruses, hacking, and online scams. In schools, homes, and workplaces in Zambia, people rely on computers and mobile phones for communication and services. Learning cyber security helps learners protect their data, avoid cybercrime, and use technology safely and responsibly.

General Competence(s):

Analytical Thinking	Ability to examine digital systems, identify weaknesses, and interpret security risks effectively.
Collaboration and Communication	Working with peers to discuss vulnerabilities, share insights, and develop security policies.
Critical Thinking and Problem-Solving	Evaluating threats and applying logical solutions to prevent or mitigate cyber-attacks.
Cybersecurity Awareness	Understanding potential cyber threats, best practices, and the importance of data protection.
Digital Literacy	Knowledge of operating systems, software, hardware, and safe digital practices.
Ethical and Societal Awareness	Recognising ethical issues and societal implications of cyber security, including privacy and responsible digital behaviour.

Hook

"Imagine receiving a text message saying you've won a lottery you never entered, asking for your mobile money PIN to claim the prize. Would you know this is a scam? How can you protect yourself and your family?" (Ask: "Has anyone here or someone they know ever received a message like that? What happened?")

Key Terms

Cyber Security: The practice of protecting computers, networks, programs, and data from digital attack, damage, or unauthorised access.

Malware: Malicious software designed to harm, exploit, or compromise a computer system (e.g., viruses, ransomware, spyware).

Phishing: A cyber-attack where attackers disguise themselves as trustworthy entities to trick individuals into revealing sensitive information.

Hacking: Gaining unauthorised access to a computer system or network.

Firewall: A network security system that monitors and controls incoming/outgoing network traffic based on security rules.

Antivirus: Software designed to detect, prevent, and remove malicious software from a computer.

Confidentiality: A principle of cyber security ensuring that data is only accessible to authorised users.

Integrity: A principle ensuring that data is accurate, complete, and has not been tampered with.

Availability: A principle ensuring that data and systems are accessible to authorised users when needed.

Two-Factor Authentication (2FA): A security process requiring two forms of identification to verify identity – adding an extra layer of protection beyond a password.

Sub-Topic 2.5.1: Introduction to Cyber Security

Introduction: This sub-topic introduces cyber security as the practice of protecting computers, networks, and data from unauthorised access, attacks, or damage. It includes understanding cyber security and its importance in digital systems, identifying vulnerabilities in operating systems, software, and hardware, and using tools and technologies to improve security. It also involves developing policies and practices that help protect data, devices, and networks from cyber threats, ensuring safe and responsible use of technology.

Specific Competence(s) – Learners will:

- 2.5.1.1 Utilise cybersecurity tools and technologies

Learning Activities

Activity 1: Discussing Cyber Security and Its Importance in Digital Systems

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for small group discussions and board presentation
- Tables or desks grouped to allow collaboration

Technology Environment:

- Projector/board to display anonymised cybercrime stories, CIA Triad diagram, and examples
- Optional computers for learners to research additional local cyber incidents

Natural Environment:

- Supportive, reflective space encouraging discussion, critical thinking, and ethical reasoning

Teaching and Learning Materials

- Projector/board for story presentation and CIA Triad diagram template for group activity
- Worksheets or activity sheets for CIA principle assignments
- Charts/posters to summarise definitions and prevention strategies / Charts with key terms for reference
- Markers, pens, and sticky notes for class discussion and group work
- Printed anonymised cybercrime scenario case studies cards (mobile money scam and social media hack)

Suggested Teaching Methods

- **Discussion-Based Learning:** Analyse real-life cybercrime cases and reflect on impacts
- **Collaborative Learning:** Groups define CIA principles, provide examples, and suggest prevention
- **Active Learning:** Groups present findings and build a class CIA Triad diagram together
- **Problem-Solving:** Identify threats and propose strategies to prevent similar incidents
- **Guided Inquiry:** Teacher facilitates linking local examples to the theoretical CIA framework

▪ How to Conduct:

Begin with a class reading of two anonymised real-world cybercrime stories relevant to Zambia: a mobile money scam (e.g., a fake Airtel/MTN text claiming a prize) and a social media account hack. After reading, ask: "Who was harmed? How were they harmed? Could it have been prevented?" Introduce the CIA Triad as the framework for thinking about what was violated: the mobile money scam violated Confidentiality (the PIN was exposed to an unauthorised person); a data breach violates Integrity (data was altered); a Denial-of-Service attack violates Availability (the system was made inaccessible). Groups of three are each assigned one CIA principle and must (a) write a one-sentence definition, (b) give a local example of a threat that would violate it, and (c) suggest one prevention strategy. Groups present their findings and the class builds a master CIA Triad diagram on the board.

□ **Content Tip:** The CIA Triad – Confidentiality, Integrity, and Availability – is the foundational framework of information security. Every cyber threat can be analysed through this lens. Confidentiality threats include data breaches and phishing (unauthorised access to private data). Integrity threats include malware that modifies files or man-in-the-middle attacks that alter data in transit. Availability threats include Distributed Denial-of-Service (DDoS) attacks that overwhelm a server and ransomware that locks users out of their own files. Grounding this in local, relatable examples is essential: mobile money scams (confidentiality), someone editing a school database to change marks (integrity), and a school's exam system going offline during tests (availability) make abstract principles tangible for learners in Zambia.

Learning Activities

Activity 2: Investigating Cyber Vulnerabilities in Operating Systems, Software, and Hardware

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for paired work with space for discussion and debrief
- Tables or desks grouped to facilitate collaboration

Technology Environment:

- Projector/board to display Chanda's profile and vulnerability categories
- Optional computers or tablets for researching safe practices

Natural Environment:

- Supportive, reflective environment encouraging critical observation, prioritisation, and ethical decision-making

Teaching and Learning Materials

- Printed fictional profile and checklist of vulnerability categories
- Worksheets or activity sheets to record identified vulnerabilities and fixes
- Projector/board for class debrief and discussion
- Charts/posters summarising digital security best practices
- Pens, markers, and sticky notes for group work

Suggested Teaching Methods

- **Case-Based Learning:** Analyse a fictional scenario to identify real-world vulnerabilities
- **Collaborative Learning:** Pairs discuss and prioritise vulnerabilities, suggesting fixes
- **Problem-Solving:** Decide which vulnerability to address first and justify reasoning
- **Guided Discussion:** Class shares findings and teacher facilitates linking to broader cybersecurity concepts
- **Active Learning:** Engage in practical decision-making about securing digital accounts and devices

How to Conduct:

Frame this activity as a "digital audit". Learners are given a fictional profile: "Chanda is a Form 2 learner. She uses one password (chanda2010) for her email, Facebook, and Zamtel account. Her phone's screen lock is off. She hasn't updated her phone's software in 8 months. She clicked a link in a WhatsApp message from an unknown number." Working in pairs, learners identify every vulnerability in Chanda's digital habits from a checklist of categories: password strength, password reuse, physical security (unlocked device), software updates, and suspicious links. Pairs then prioritise the vulnerabilities from most to least dangerous and recommend a specific fix for each. The class debriefs by discussing: "Which vulnerability would you fix first if you could only do one? Why?"

□ **Content Tip:** Think of cyber security like securing a home. A weak password is an unlocked door. Password reuse is using the same key for your house, car, and workplace – if one is stolen, everything is compromised. Outdated software is a broken window – attackers know about the gap and will exploit it before you fix it. An unlocked phone is leaving your house door wide open. Clicking unknown links is letting a stranger walk in. Software updates are not cosmetic changes; they are security patches that close known vulnerabilities. A practical metric: the National Institute of Standards and Technology (NIST) reports that over 80% of successful cyber-attacks exploit either weak passwords, password reuse, or unpatched software – all of which are free to fix with good habits. Facilitating a learner self-audit of their own digital habits ("do you use the same password for multiple accounts?") makes the lesson personally relevant and motivating.

Learning Activities

Activity 3: Using Cyber Security Tools and Technologies

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom organised into three clearly defined stations for rotation
- Seating arranged in small groups to allow smooth movement between stations

Technology Environment:

- Computers with internet access for password testing
- Antivirus software installed or screenshots available
- Projector/board to display instructions and key concepts

Natural Environment:

- Interactive and engaging environment promoting hands-on exploration, curiosity, and reflective thinking

Teaching and Learning Materials

- Computers with internet access (password strength testing) for Station 1
- Antivirus software installed or demo screenshots for Station 2
- Printed 2FA role-play cards and numbered code cards
- Worksheets for recording observations (password strength, antivirus features, 2FA process)
- Charts/posters showing characteristics of strong passwords and 2FA concepts
- Pens, markers, and sticky notes

Suggested Teaching Methods

- **Station Rotation Method:** Structured movement between password testing, antivirus exploration, and 2FA simulation
- **Experiential Learning:** Direct interaction with tools to understand cybersecurity concepts
- **Collaborative Learning:** Group work to analyse results and share findings
- **Demonstration:** Teacher models use of antivirus tools and explains 2FA
- **Reflective Discussion:** Learners analyse outcomes and discuss security implications

How to Conduct:

Structure this as a three-station practical rotation (10 minutes per station, groups rotate). Station 1 – Passwords: Learners use an online password strength meter (e.g., howsecureismypassword.net) to test three passwords of increasing complexity (e.g., "cat", "Catfish12", "C@t!sh#2026Zambia"). They record the time-to-crack estimate for each and identify what elements of a strong password (length, uppercase, lowercase, numbers, symbols) made the difference. Station 2 – Antivirus: Learners explore the interface of an installed antivirus program (or a demo screenshot). They identify: the last scan date, what to do if a threat is found, and how to schedule automatic scans. Station 3 – 2FA Simulation: Using printed scenario cards, learners role-play a 2FA login: they "enter a password" (read a card), then "receive an SMS code" (teacher hands them a numbered card), and must use both to "log in". Debrief: what happens if only one factor is compromised?

□ **Content Tip:** A strong password should be at least 12 characters long and include a mix of uppercase letters, lowercase letters, numbers, and special symbols. The single most impactful improvement, however, is length: every additional character increases the time required for a brute-force attack exponentially. A 6-character password can be cracked in seconds; a 16-character passphrase takes billions of years. A passphrase (e.g., "Zambezi!River2026") is both strong and memorable. Two-Factor Authentication (2FA) adds a second layer: even if an attacker steals the password, they cannot log in without the second factor (a code sent to the user's phone). This is why 2FA is the single most effective security measure an individual can enable. Real-world connection: Airtel Money, MTN MoMo, and most banks use 2FA via SMS codes – learners are likely already using it without knowing the term.

Learning Activities

Activity 4: Developing Policies to Protect Data, Devices, and Networks from Cyber Threats

Learning Environment Setup

Artificial (Physical – Classroom) Environment

- Arrange desks in small collaborative groups of 4–5 learners to support discussion and drafting.
- Provide wall space or board space where groups can display their drafted AUP rules.
- Create a “policy presentation corner” where groups present and defend their rules.
- Display the final consolidated AUP visibly in the classroom or computer lab.

Technology Environment

- Use a projector to display a sample Acceptable Use Policy for comparison.
- If available, allow learners to type the final version using a word processor for printing.
- Use a digital voting tool (or simple online poll) if computers are accessible.
- Demonstrate screen-locking and password protection on an actual computer for practical understanding.

Natural Environment

- Connect the activity to learners' real-life digital experiences (school lab, home devices, internet cafés).
- Encourage learners to reflect on real situations where cyber rules were broken and consequences occurred.
- Use the actual computer lab as the display location for authenticity and ownership.

Teaching and Learning Materials

- Manila paper and markers for drafting rules.
- Sample Acceptable Use Policy (AUP) from another school for reference.
- Voting slips for consolidation and selection of best rules.
- Sticky notes for peer feedback.
- Projector/computer for demonstrating password protection and screen locking.
- Printed template showing: Rule (Do), Explanation, Consequence format.

Suggested Teaching Methods

- **Problem-Based Learning:** Present the task as a real request from the headteacher to give authenticity and purpose.
- **Collaborative Learning:** Learners work in groups to draft and refine rules together.
- **Discussion Method:** Facilitate class discussion to analyse and improve drafted rules.
- **Demonstration Method:** Show practical examples of locking screens and managing passwords.
- **Peer Review:** Groups evaluate each other's rules before voting.
- **Participatory Decision-Making:** Learners vote and co-create the final consolidated AUP.
- **Reflective Learning:** Guide learners to explain why each rule exists and its consequences.

▪ How to Conduct:

Frame this as a real brief: "The headteacher has asked your class to create the school's first Computer Lab Acceptable Use Policy (AUP). This will be printed and displayed in the lab, and all learners must sign it." Learners work in groups to draft five to seven rules covering: password management (no sharing), software use (no downloading unapproved software), physical security (locking screens when leaving), reporting (what to do if they see a threat or inappropriate content), and privacy (not accessing other learners' accounts). Groups must write each rule in both positive form ("Do: Lock your screen when you leave the computer") and explain the consequence of not following it ("Unlocked screens allow others to access your account and private data"). Groups share their drafts, the class votes on the best rules, and a final consolidated policy is created together and displayed in the lab.

□ **Content Tip:** A well-designed Acceptable Use Policy (AUP) is a foundational security document for any organisation – from a school computer lab to a multinational company. Effective AUPs share three characteristics: they are specific ("Use a password of at least 8 characters" is better than "Use a strong password"), they are enforceable (there must be a realistic way to check compliance), and they include consequences (learners understand why each rule exists). Positive framing ("Do" before "Don't") is more effective psychologically. The creation process itself is pedagogically valuable: when learners write the rules, they own them and are more likely to follow them. Connect this to the broader concept of cyber security culture: technology alone cannot secure a system – human behaviour is always the final layer of defence.

Expected Standard

Cybersecurity tools and technologies utilised accordingly.

ASSESSMENT

Examples of Formative

Scenario: Your grandmother receives a text message: "Grandma, it's me your grandson. I have lost my phone. This is my new number. Please send me K200 for data so I can call you later."

1. (Identifying Threats) What type of cyber threat is this message an example of?
2. (Policy Development) What are two specific pieces of advice you would give your grandmother to help her avoid falling for this kind of trick in the future?

Formative Rubric:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Understanding Cyber Threats	Can explain different types of threats (phishing, malware, hacking) and give real-world examples.	Identifies common cyber threats like phishing, malware, and hacking.	Recognises some threats but confuses them (e.g., calls all threats 'viruses').	Cannot identify common cyber threats.
Identifying Vulnerabilities	Proactively identifies multiple vulnerabilities in a scenario and suggests solutions.	Can identify basic vulnerabilities (weak passwords, no firewall).	Identifies only obvious vulnerabilities with prompting.	Cannot identify any vulnerabilities.
Using Security Tools	Demonstrates proficient use of tools (e.g., sets up 2FA, configures strong passwords) independently.	With guidance, can set a strong password and explore antivirus software.	Attempts to use tools but makes errors (e.g., weak password).	No attempt or unable to use any security tool.
Policy Development	Creates a comprehensive, realistic policy with clear rules and justifications.	Creates a basic policy with 3–5 relevant rules.	Policy rules are vague or not practical.	Does not participate or produces no policy.

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Terminology Knowledge	Accurately uses terms like confidentiality, integrity, 2FA in discussions.	Can define most key terms correctly.	Confuses some terms (e.g., confidentiality vs. integrity).	Cannot define basic cyber security terms.

Example of Summative Assessment:

Scenario: Your school wants to run a workshop for parents and local vendors on “Protecting Your Mobile Money from Scams.” Your class has been asked to create a simple information sheet.

Task – Policy Creation Project:

Create a simple, one-page guide titled “Stay Safe: Advice for Mobile Money Users” (or its equivalent in your local language). Your guide must include:

1. A definition of a “scam” in simple terms. A clear, realistic example of a mobile money scam.
2. Two rules for creating a strong PIN for a mobile money account.
3. An explanation of why the PIN should never be shared, not even with someone claiming to be from Airtel, MTN, or Zamtel. One final piece of advice: “If it sounds too good to be true (like winning a lottery you never entered), it’s probably a scam.”

SUMMATIVE RUBRIC:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Conceptual Knowledge (Test)	Scores 90–100%, demonstrating deep understanding of CIA triad and threat landscape.	Scores 70–89%, correctly answering most questions.	Scores 50–69%, with gaps in understanding key concepts.	Scores below 50%, indicating minimal understanding.
Practical Tool Use	Successfully demonstrates multiple security measures (strong password, 2FA, antivirus scan) independently.	Successfully demonstrates at least one tool (e.g., creates a strong password).	Attempts demonstration but requires significant help to complete.	Unable to demonstrate any tool usage.
Policy Creation Project	Policy is thorough, addresses multiple scenarios, includes enforcement suggestions, and is well-presented.	Policy includes clear, relevant rules for a school computer lab.	Policy is incomplete or contains irrelevant/impractical rules.	Policy is missing or not submitted.
Application to Real Life	Provides insightful examples of how they would apply cyber security at home and school.	Can give a simple example of applying cyber security in daily life.	Struggles to connect concepts to personal experience.	Cannot apply concepts to real life.

Summary – Key Points Recap:

- Cyber security protects digital systems from unauthorised access and attacks.
- Cyber threats like phishing and scams affect both individuals and institutions in Zambia.
- Simple tools (antivirus, firewalls) and good habits (strong passwords, updates) improve security.
- Everyone has a role in staying safe online.
- Prevention is better than cure when dealing with cyber threats.

TOPIC 2.6: ALGORITHMS

Overview

This topic introduces learners to the fundamental principles of algorithms. Learners will explore how to design, represent, and refine step-by-step procedures to solve computational and real-world problems. Emphasis is placed on algorithmic thinking, logic development, and the use of visual tools like flowcharts and pseudocode.

General Competence(s):

Analytical Thinking	Ability to break down problems and understand the steps required to form algorithms.
Algorithmic Thinking	Ability to design step-by-step procedures and logical sequences to solve problems efficiently.
Critical Thinking and Problem-Solving	Evaluating different algorithm solutions and selecting the most effective approach.
Creativity and Innovation	Developing unique and efficient algorithms and improving existing solutions.
Digital Literacy	Understanding how algorithms are applied in digital systems and computing environments.
Technical Proficiency	Applying skills in flowcharts, pseudocode, and algorithm implementation accurately.

Hook

"What if you could write a recipe for a computer to solve any problem – from sorting your playlist to finding the shortest route to school? How do you think computers 'think' step-by-step?" (Start with: "How would you write instructions for someone to make nshima?" Have a volunteer follow the instructions literally to highlight the need for precision.)

Key Terms

Algorithm: A precise, step-by-step, finite set of instructions designed to perform a specific task or solve a problem.

Flowchart: A visual diagram that represents an algorithm using standard symbols (ovals, rectangles, diamonds) to show the flow of steps and decisions.

Pseudocode: A simplified, informal way of writing an algorithm's steps using plain language and basic programming structures, without strict syntax rules.

Sequence: The logical order in which steps in an algorithm are executed, one after another.

Selection (Decision): A control structure that chooses between different paths based on a condition (e.g., "IF it's raining, THEN take an umbrella").

Iteration (Loop): A control structure that repeats a block of steps until a specific condition is met.

Optimisation: The process of refining an algorithm to make it more efficient (e.g., reducing the number of steps or memory used).

Input/Output: Data received by an algorithm (input) or produced by it (output).

Process: A single step or operation within an algorithm that transforms data.

Efficiency: A measure of how well an algorithm uses resources like time and memory.

Sub-Topic 2.6.1: Fundamentals of Algorithms

Introduction: This sub-topic introduces algorithms as step-by-step instructions that solve problems. Teachers should explain the different types of algorithms and discuss their key characteristics, helping learners understand how algorithms work in computing. Guide learners to design flowcharts and write pseudocode, showing how algorithms can be applied to solve computational problems. Encourage learners to refine and optimise their algorithms to improve efficiency and accuracy. Introduce various algorithm design techniques and provide practical examples so learners can practice creating and applying algorithms effectively.

Specific Competence(s) – Learners will:

- 2.6.1.1 Design algorithms

Learning Activities

Activity 1: Identifying Different Types of Algorithms

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for interactive discussion and pair work
- Open space at the front for demonstration (role-play of the nshima-making algorithm)

Technology Environment:

- Projector/board to display examples of algorithms (recipes, routines, GPS routes)
- Optional digital devices for viewing additional examples

Natural Environment:

- Engaging and relaxed environment encouraging participation, curiosity, and humour-driven learning

Teaching and Learning Materials

- Algorithm classification cards (recipe, morning routine, GPS route)
- Nshima ingredients for the literal interpretation activity
- Board/flip chart for writing instructions and classifications
- Worksheets for classification activity / Chalkboard or whiteboard for the class debrief
- Charts/posters showing types of algorithms (sequential, selection, iteration)
- Everyday object examples (traffic light, ATM process illustrations)

Suggested Teaching Methods

- **Demonstration:** Role-play to show importance of precise instructions in algorithms
- **Discussion-Based Learning:** Class discussion on everyday algorithms and their importance
- **Collaborative Learning:** Pair work to classify types of algorithms
- **Inquiry-Based Learning:** “Spot the algorithm” activity to explore real-life applications
- **Active Learning:** Learners engage through participation, observation, and classification

▪ How to Conduct:

Open with the nshima-making algorithm from the hook. Ask a volunteer to give you instructions for making nshima, and follow them literally and robotically (e.g., if they say “add water”, ask “how much water? Where is the water? Which hand do I use?”). This reliably generates laughter while powerfully demonstrating that an algorithm must be precise and unambiguous. Then transition to more computational examples. Present three algorithm cards: a recipe (sorting algorithm), morning routine (sequential algorithm), and GPS route (pathfinding algorithm). In pairs, learners classify each as primarily sequential (steps in order), selection-based (requires decisions), or iteration-based (repeats steps). Conclude with a “spot the algorithm” challenge: identify hidden algorithms in everyday objects – a traffic light timer, a vending machine payment sequence, an ATM withdrawal process.

□ **Content Tip:** An algorithm is defined by four essential characteristics: Finiteness (it must stop – an algorithm that runs forever is not an algorithm), Definiteness (each step must be clear and unambiguous – "add some water" is not definite; "add 500ml of water" is), Input/Output (it takes in data and produces a result), and Effectiveness (every step must be doable with the available resources). The nshima/sandwich activity is pedagogically powerful precisely because it fails on definiteness – the instructions are ambiguous. Use this failure productively: "What did you need to add to make the instruction unambiguous?" The answer is the definition of algorithmic thinking: specify the exact action, the exact object, and the exact condition.

Learning Activities

Activity 2: Discussing Characteristics of Algorithms

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for group work and peer evaluation
- Board space for displaying instruction sets and class discussion

Technology Environment:

- Projector/board to present examples of algorithms and characteristics
- Optional computers for typing and reviewing algorithms

Natural Environment:

- Supportive environment encouraging analysis, critique, and constructive feedback

Teaching and Learning Materials

- Prepared instruction sets/examples (good and flawed algorithms)
- Worksheets for evaluating algorithm characteristics
- Charts/posters showing the four characteristics (finiteness, definiteness, input/output, effectiveness)
- Writing materials (exercise books, pens, markers)
- Board/flip chart for class discussion
- Whiteboard or flipchart with four example instruction sets
- Algorithm evaluation worksheet (finiteness, definiteness, input/output, effectiveness)
- Everyday task cards for the group swap activity

Suggested Teaching Methods

- **Discussion-Based Learning:** Analyse and evaluate given algorithm examples
- **Collaborative Learning:** Groups create and assess algorithms together
- **Peer Assessment:** Groups exchange and evaluate each other's algorithms
- **Problem-Solving:** Identify errors and propose improvements to algorithms
- **Guided Inquiry:** Teacher facilitates linking examples to key characteristics

How to Conduct:

Present four instruction sets on the board and ask learners to evaluate each against the four algorithm characteristics (finiteness, definiteness, input/output, effectiveness). Example 1: a good algorithm (clear steps with a definite end). Example 2: an infinite loop ("Keep counting up from 1" – violates finiteness). Example 3: ambiguous steps ("Sort the pile somehow" – violates definiteness). Example 4: an impossible step ("Divide by zero" – violates effectiveness). For each example, learners identify which characteristic(s) are violated and propose a fix. Consolidate with a group task: each group writes a five-step algorithm for an assigned everyday task (boiling water for tea, crossing a road safely, paying at a till) and then swaps with another group to evaluate their algorithm against all four characteristics. Groups provide written feedback identifying any violations.

□ **Content Tip:** A common misconception to address directly: a long set of instructions is not necessarily an algorithm, and a short one is not necessarily simple. What makes something an algorithm is the combination of all four characteristics working together. For example, "Think of a number between 1 and 10" is a valid algorithm step (it has finite options and a clear output), while "Do your best" is not (it is not definite – what does 'best' mean in computational terms?). The concept of definiteness is especially important to connect forward to programming: in Python, for example, the computer executes exactly what is written, not what you meant to write. Imprecise thinking at the algorithm stage leads directly to buggy code at the programming stage.

Learning Activities

Activity 3: Designing Flowcharts and Writing Pseudocode

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for individual and group work, with wall space for the "symbol gallery"
- Clear board space for teacher demonstration of flowcharts

Technology Environment:

- Projector/board to display flowchart examples and pseudocode structures
- Optional computers for creating digital flowcharts

Natural Environment:

- Structured and supportive environment promoting logical thinking, creativity, and independent practice

Teaching and Learning Materials

- Flowchart symbol cards (oval, rectangle, diamond, arrows) displayed as a gallery
- Worksheets for flowchart drawing and pseudocode writing
- Graph paper, pencils, rulers, and markers
- Charts/posters showing standard flowchart symbols and pseudocode conventions
- Board/flip chart for demonstration
- Laminated flowchart symbol reference cards for each learner
- Graph paper or flowchart templates for neat diagram drawing
- Access to draw.io or Lucidchart for digital flowcharts

Suggested Teaching Methods

- **Demonstration:** Teacher models flowchart design and pseudocode writing using think-aloud strategy
- **Guided Practice:** Learners follow along to create a sample flowchart
- **Independent Practice:** Learners design their own flowcharts and write corresponding pseudocode
- **Visual Learning:** Use of symbol gallery to reinforce understanding of flowchart components
- **Problem-Solving:** Apply logic to real-life scenarios using flowcharts and pseudocode

How to Conduct:

Introduce flowchart symbols with a "symbol gallery" – post large cards on the wall, each showing one symbol, its name, and its function. Then demonstrate designing a flowchart live for a familiar process: "Checking if a number is even or odd." Think aloud while drawing: "I start with an oval labelled START. Then a rectangle: INPUT a number. Then a diamond: Is number $\div$ 2 remainder = 0? If YES, a rectangle: OUTPUT 'Even'. If NO, a rectangle: OUTPUT 'Odd'. Then an oval: END." Learners copy this, then independently design a flowchart for: checking if a learner has passed an exam (score $\geq$ 50), and deciding whether to go to school (Is it raining? Do I have an umbrella?). The parallel pseudocode task: after drawing the flowchart, learners translate it into structured plain-English pseudocode, using IF/THEN/ELSE for decisions and FOR/WHILE for loops.

□ **Content Tip:** Standard flowchart symbols carry precise meanings that should be taught and enforced from the start: Oval (Terminator): used only for START and END. Rectangle (Process): any action or calculation step (e.g., Set Total = Price × Quantity). Diamond (Decision): a question with exactly two outcomes, YES/NO or TRUE/FALSE. Parallelogram (Input/Output): receiving data from a user (INPUT) or displaying a result (OUTPUT). Arrows: show the direction of flow and must always have arrowheads. A diagram without these conventions is not a flowchart – it is a sketch. Enforcing correct symbol usage from the beginning builds professional precision and prepares learners for qualifications where incorrect symbols lose marks. Provide a laminated symbol reference card for learners to keep in their notebooks.

Learning Activities

Activity 4: Applying Algorithms to Computational Problems

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for individual work and small group support
- Adequate board space for step-by-step demonstrations and worked examples

Technology Environment:

- Projector/board to demonstrate flowcharts, pseudocode, and trace tables
- Optional computers for typing and testing algorithm logic

Natural Environment:

- Structured environment encouraging logical reasoning, persistence, and analytical thinking

Teaching and Learning Materials

- Problem sheets with sequential, selection, and iteration tasks
- Flowchart templates for algorithm design
- Trace table templates (variable columns, iteration rows) for step-by-step verification
- Exercise books, pens, rulers, and graph paper
- Charts/posters showing control structures (sequence, selection, iteration)
- Problem sheets with the three-level computational challenges

Suggested Teaching Methods

- **Demonstration:** Teacher models solving problems using flowcharts, pseudocode, and trace tables
- **Guided Practice:** Learners work through Level 1 and Level 2 tasks with support
- **Independent Practice:** Learners solve Level 3 problems using loops and trace tables
- **Problem-Solving Approach:** Apply algorithms to real computational problems
- **Analytical Learning:** Use trace tables to test and verify algorithm correctness

How to Conduct:

Progress through three problem levels. Level 1 – Sequential: Design an algorithm to calculate a learner's average score from five subjects. Learners write both a flowchart and pseudocode, using the INPUT, PROCESS (SUM ÷ 5), and OUTPUT structure. Level 2 – Selection: Design an algorithm to award a grade (A, B, C, D, F) based on a score. This requires nested IF/ELSE IF conditions. Learners trace the algorithm with three different test inputs (95, 65, 35) to verify it produces the correct grade for each. Level 3 – Iteration: Design an algorithm to find the highest score in a list of 30 learners' marks. This requires a loop with a running maximum variable. Introduce the "trace table" technique: a table with columns for each variable (score, max, counter) that learners fill in manually for the first five iterations to verify the algorithm's logic before trusting the result.

□ **Content Tip:** The three fundamental control structures of all algorithms (and all programming languages) are Sequence (steps executed in order), Selection (IF condition THEN action ELSE alternative action), and Iteration (WHILE condition DO steps / FOR i FROM 1 TO n DO steps). Any computational problem can be solved using combinations of these three structures. The trace table technique is an essential but underused skill: manually stepping through an algorithm with specific inputs to verify it produces the correct outputs before implementing it in code. It is the algorithm-level equivalent of unit testing in professional software development. When an algorithm fails a trace table, the error is found and fixed at the logical level – which is far faster than debugging compiled code.

Learning Activities

Activity 5: Refining and Optimising Algorithms

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for pair and group work to encourage discussion and comparison
- Board space for illustrating algorithm steps and comparisons

Technology Environment:

- Projector/board to display linear and binary search processes
- Optional computers for simulating searches and tracing algorithms

Natural Environment:

- Analytical environment promoting critical thinking, evaluation, and logical reasoning

Teaching and Learning Materials

- Comparison sheets (Linear Search vs Binary Search)
- Sample algorithms with intentional logical errors for the debugging task
- Trace table templates for algorithms for step-by-step analysis
- Worksheets for recording observations and conclusions
- Charts/posters showing concepts of efficiency (time and steps)
- Writing materials (exercise books, pens, rulers)
- Side-by-side algorithm comparison sheets (Linear Search vs. Binary Search)

Suggested Teaching Methods

- **Comparative Analysis:** Learners compare two algorithms solving the same problem
- **Guided Discovery:** Teacher leads learners to observe differences in efficiency
- **Collaborative Learning:** Pairs/groups trace algorithms and discuss findings
- **Problem-Solving:** Identify, analyse, and correct logical errors in algorithms
- **Analytical Learning:** Use trace tables to evaluate performance and correctness

How to Conduct:

Present two algorithms that solve the same problem: searching a list of 100 names for a specific name. Algorithm A (Linear Search): starts at the first name and checks every name one by one until the target is found – in the worst case, 100 comparisons. Algorithm B (Binary Search): requires the list to be sorted first, then repeatedly halves the search space – in the worst case, only 7 comparisons for a 100-item list ($\log_2 100 \approx 7$). Learners trace both algorithms with a small 10-name sorted list to observe the difference. Discuss: "Is Binary Search always better? What is its limitation?" (It requires a pre-sorted list.) Present a third task: an algorithm with a logical error (e.g., a loop that counts to 11 instead of 10, or an IF condition that uses ">" instead of "≥", causing off-by-one errors). Learners identify and fix the error and explain what the consequence of the bug would have been in a real system.

□ **Content Tip:** Algorithm efficiency is measured in terms of time complexity (how many steps does it take as the input size grows?) and space complexity (how much memory does it use?). At this level, learners need not study Big-O notation formally, but developing the intuition for efficiency is important. The Linear Search vs. Binary Search comparison is an ideal intuition-builder: doubling the list size doubles the steps for linear search, but only adds one step for binary search. This exponential difference becomes enormous at scale – searching a list of one million names takes up to 1,000,000 steps linearly but at most 20 steps with binary search. The practical lesson: choosing the right algorithm for the problem is often more impactful than having the fastest computer.

Learning Activities

Activity 6: Using Different Algorithm Design Techniques

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for group work and presentations

Technology Environment:

- Projector/board to demonstrate examples of algorithm design techniques
- Optional computers for designing and comparing solutions

Natural Environment:

- Interactive environment encouraging discussion, creativity, and real-life problem-solving

Teaching and Learning Materials

- Problem context cards for Brute Force, Divide and Conquer, and Greedy technique activities
- Flowchart templates and pseudocode reference guides
- Example comparison sheets for different techniques, Charts/posters explaining algorithm design techniques, Writing materials (exercise books, pens, markers)

Suggested Teaching Methods

- **Collaborative Learning:** Groups solve problems using assigned algorithm design techniques
- **Problem-Based Learning:** Real-life scenarios used to apply different techniques
- **Demonstration:** Teacher explains and models each design technique
- **Comparative Discussion:** Class evaluates effectiveness of different approaches
- **Presentation Method:** Groups present solutions and justify chosen techniques

How to Conduct:

Introduce three design techniques through real Zambian problem contexts. Brute Force: "Find the cheapest combination of items at a market that totals exactly K50." Explain that brute force checks every possible combination – it is simple to implement but slow for large inputs. Divide and Conquer: "You have an alphabetically sorted list of 256 names. Find 'Mwamba'." Demonstrate binary search as a classic divide-and-conquer algorithm – split the list in half, decide which half contains the target, and repeat. Greedy Algorithm: "A minibus is taking on passengers at a bus stop. It seats 14. Passengers board largest-group-first to fill the seats most efficiently." A greedy algorithm always makes the locally optimal choice at each step. Groups solve one problem using their assigned technique, present their flowchart/pseudocode to the class, and the class discusses: "Would a different technique solve this problem? Would it be faster or slower?"

□ **Content Tip:** Design technique selection is as important as the algorithm itself. Brute Force is simple and always finds the correct answer, but is only practical for small input sizes. Divide and Conquer (binary search, merge sort) dramatically reduces the number of comparisons needed by repeatedly halving the problem space – it is efficient for large, sorted datasets. Greedy algorithms make the locally optimal choice at each step and are fast and simple, but do not always find the globally optimal solution (a classic example: the coin-change problem with unusual denominations). Connecting these techniques to real contexts in Zambia – bus scheduling, market pricing, patient queue management at a clinic – grounds abstract computer science in the learners' lived experience and demonstrates the transferable value of computational thinking.

Expected Standard

Algorithms designed correctly.

ASSESSMENT

Examples of Formative

Scenario: Your teacher asks you to write instructions for a learner from a rural area who has never used an ATM to withdraw K200 from a bank like Zanaco or Stanbic.

1. (Algorithmic Thinking) Write a precise, step-by-step algorithm for this task. Start from “Insert your card” and end with “Take your card, receipt, and cash.” Make sure the steps are in the correct sequence.
2. (Flowchart Design) Take your algorithm and add a decision point. At some stage, the ATM will ask if you want a receipt. Your flowchart should show two paths: “Yes, print receipt” and “No, no receipt”, before both paths join back to the final step.

Formative Rubric:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Understanding of Algorithms	Clearly explains what an algorithm is and can identify them in everyday life.	Can define an algorithm and give a simple everyday example.	Has a vague idea but struggles to give examples.	Cannot define or identify an algorithm.
Flowchart Design	Designs accurate, well-structured flowcharts for complex problems using correct symbols.	Designs correct flowcharts for simple tasks with all symbols used appropriately.	Flowcharts have missing or misused symbols; some steps unclear.	Cannot create a flowchart.
Pseudocode Writing	Writes clear, structured pseudocode that could be easily translated into code.	Writes understandable pseudocode for simple algorithms.	Pseudocode is disorganised or difficult to follow.	Cannot write pseudocode.
Application of Constructs	Skilfully uses sequence, selection, and iteration in designs.	Uses sequence and at least one selection or iteration correctly.	Attempts to use selection/iteration but with logical errors.	Only uses sequence; no decisions or loops.
Peer Review Participation	Provides insightful feedback and asks probing questions during walkthroughs.	Listens to peers and offers basic feedback.	Passive during peer review; does not contribute.	Disruptive or disengaged.

Example of Summative Assessment:

Scenario (Project: “The Efficient Daily Task”): A security guard at a secondary school in Chipata needs to check that all 12 classroom doors are locked at the end of the day. He currently walks to each door in a random order, often passing the same doors multiple times and wasting time.

Task:

1. Flowchart: Design a flowchart for a more efficient algorithm. Assume classrooms are in two straight lines. Your algorithm should start at the first classroom, check if it's locked, and then move to the next one until all 12 are checked.
2. Pseudocode: Write the pseudocode for the same process. Use a loop (e.g., FOR each room FROM 1 TO 12) to represent the repetition.
3. Refinement Report: In a short paragraph, explain why your algorithm is more efficient than the guard's random method. Think about time saved and how it ensures he never misses a classroom.

SUMMATIVE RUBRIC:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Flowchart Correctness	Flowchart is fully correct, uses all symbols properly, and is easy to follow.	Flowchart correctly represents the algorithm with minor errors.	Flowchart has significant errors or missing steps.	Flowchart is incorrect or not attempted.
Pseudocode Clarity	Pseudocode is precise, well-structured, and mirrors the flowchart perfectly.	Pseudocode is understandable and follows the algorithm.	Pseudocode is ambiguous or does not match the flowchart.	Pseudocode is missing or incomprehensible.
Refinement Report	Provides a detailed analysis of inefficiencies and explains how the algorithm was improved.	Identifies at least one improvement made to the algorithm.	Mentions improvement but does not explain why it is better.	No refinement or report.
Timed Task Performance	Completes the timed task accurately and efficiently within the time limit.	Completes the task correctly within the time limit.	Completes the task but with errors or exceeds time limit.	Does not complete the task.

Summary – Key Points Recap:

- An algorithm is a precise, step-by-step procedure to solve a problem.
- A valid algorithm must be well-defined, finite, and produce consistent results.
- Algorithms are built from three fundamental control structures: Sequence, Selection (if-else), and Iteration (loops).
- Flowcharts are visual diagrams and pseudocode is structured plain-English text used to represent algorithms.
- Algorithmic thinking is a critical skill for breaking down complex problems.
- Refining/Optimising an algorithm improves its correctness, clarity, and efficiency.

TOPIC 2.7: WEB DEVELOPMENT

Overview

Web development involves the creation and styling of digital content to ensure it is functional, accessible, and visually engaging. This topic focuses on Cascading Style Sheets (CSS), the essential design tool that allows developers to control the presentation of HTML structures.

General Competence(s):

Analytical Thinking	Analysing design requirements and selecting appropriate CSS methods and properties.
Collaboration and Communication	Working with others to design, review, and improve web page layouts.
Critical Thinking and Problem-Solving	Identifying and correcting layout and styling issues in web pages.
Creativity and Innovation	Designing attractive and user-friendly web pages using CSS styling techniques.
Digital Literacy	Understanding how CSS integrates with HTML to create styled web content.
Ethical and Societal Awareness	Ensuring accessibility and inclusive web design for all users.
Technical Proficiency	Applying CSS rules, selectors, and properties accurately.
Web Development Skills	Developing web pages using CSS concepts such as selectors, box model, and layouts.

Hook

"Why do two different websites look completely different even if they are both about the same topic? Who decides the colours and fonts?" (Show a plain HTML page with no styling, then apply a CSS file and transform its appearance completely. The contrast is very powerful.)

Key Terms

CSS (Cascading Style Sheets): A language used to describe the visual presentation of a web page, controlling layout, colours, fonts, and more.

CSS Selector: A pattern used to select and target specific HTML elements to style (e.g., h1, .className, #idName).

Property: A specific style attribute to change, such as color, font-size, or margin.

Value: The setting assigned to a CSS property (e.g., setting color to blue).

Inline CSS: CSS applied directly to a single HTML element using the style attribute.

Internal CSS: CSS placed within a <style> tag in the page's header.

External CSS: CSS in a separate .css file linked to an HTML page – best practice for multi-page sites.

Box Model: A CSS concept describing every element as a rectangular box with content, padding (space around content), a border, and a margin (space outside the border).

Responsive Design: A web design approach that makes pages render well on a variety of devices and screen sizes.

Sub-Topic 2.7.1: Cascading Style Sheets (CSS) Concepts

Introduction: This sub-topic introduces Cascading Style Sheets (CSS) and how they are used to style web pages. Teachers should guide learners to identify the different types of CSS: inline, internal, and external, and explain the basic structure of a CSS rule. Learners should practise using different CSS elements and implementing the three methods to apply styles. Teach how to use text properties to style fonts, manage the box model, and apply background properties. Emphasise the importance of web accessibility and guide learners to evaluate their designs to ensure they are clear and usable for all users.

Specific Competence(s) – Learners will:

- 2.7.1.1 Apply CSS concepts to websites.

Learning Activities

Activity 1: Identifying the Different Types of CSS: Inline, Internal, and External

Learning Environment Setup

Artificial (Physical – Classroom):

- Computer lab with individual workstations for learners
- Space for learners to implement CSS on HTML templates and share screens

Technology Environment:

- Computers with text editor (VS Code, Notepad++) and web browser
- Projector and screen to demonstrate CSS transformations

Natural Environment:

- Visual and interactive environment encouraging experimentation with style and structure
- Posters or charts showing HTML structure vs CSS styling for reference

Teaching and Learning Materials

- Computers with a text editor (VS Code or Notepad++) and a web browser
- Plain HTML template ("Tourism Zambia" page)
- Pre-written external CSS file to demonstrate transformation
- Printed reference sheets comparing Inline, Internal, and External CSS
- Projector/board for teacher demonstration
- Writing materials (exercise books, pens, markers)
- A plain HTML template file ("Tourism Zambia" page)
- A pre-written external CSS file to demonstrate the transformation

Suggested Teaching Methods

- **Demonstration:** Teacher shows plain HTML, then applies all three CSS methods to highlight differences
- **Guided Practice:** Learners implement inline, internal, and external CSS on their templates
- **Think-Pair-Share:** Learners discuss in pairs which CSS method is most efficient and why
- **Hands-on Experimentation:** Learners modify colors, fonts, and styles to observe effects
- **Class Discussion:** Reflect on best practices and maintainability of CSS methods

How to Conduct:

Begin with the hook demonstration: load a plain HTML page (text only, no CSS) and ask learners to describe it ("boring", "plain", "looks like a document"). Then, link a CSS stylesheet and reload – the transformation should be dramatic. Explain: "HTML built the house. CSS painted it, furnished it, and put in the garden." Present the same simple heading styled three ways on the board: (1) Inline: `<h1 style="color:red;">Welcome</h1>`; (2) Internal: a `<style>` block in the HTML `<head>`; (3) External: a link to `styles.css`. For each method, ask: "If this page had 50 headings, how long would it take to change all of them to blue?" The answer reveals why external CSS is best practice: one change in the `.css` file updates all 50 headings across all pages of the site simultaneously. Learners then implement all three methods on a simple "Tourism Zambia" HTML template.

□ **Content Tip:** The relationship between HTML and CSS is one of the most important architectural concepts in web development: HTML defines the structure and semantic meaning of content (this is a heading, this is a paragraph, this is an image), while CSS defines the visual presentation (this heading is purple, this paragraph has 16px text, this image has a 2px border). This separation of concerns is a design principle that makes websites easier to maintain, update, and adapt. For a concrete demonstration of its power: the CSS Zen Garden website (csszengarden.com) shows dozens of radically different web designs all using exactly the same HTML file but different CSS stylesheets – making the point that structure and style are genuinely independent.

Learning Activities

Activity 2: Describing the Basic Structure of a CSS Rule

Learning Environment Setup

Artificial (Physical – Classroom):

- Computer lab with individual workstations for learners
- Space for learners to compare HTML and CSS files side by side

Technology Environment:

- Computers with modern web browsers (Chrome, Firefox) and text editor
- Projector/board to demonstrate CSS rules using Developer Tools

Natural Environment:

- Interactive and investigative environment encouraging learners to explore live CSS effects
- Visual displays or charts showing CSS rule anatomy for reference

Teaching and Learning Materials

- Computers with a modern web browser (Chrome or Firefox for Developer Tools) and text editor
- Pre-written HTML and CSS files with at least six labelled rules
- CSS rule anatomy table worksheet (Selector | Property | Value | Effect)
- Projector/board for teacher demonstration of Developer Tools
- Writing materials (exercise books, pens, markers)

Suggested Teaching Methods

- **Demonstration:** Teacher models how to use Developer Tools to inspect CSS rules
- **Guided Practice:** Learners identify selectors, properties, values, and effects for given rules
- **Hands-on Activity:** Learners write three new CSS rules from scratch and explain components
- **Collaborative Discussion:** Pairs compare findings and clarify common errors
- **Analytical Thinking:** Learners predict how changing a property or value will alter the page

How to Conduct:

Provide learners with a pre-written HTML page and a CSS file containing at least six rules. Learners open both files in the text editor and the browser simultaneously. Using the browser's built-in Developer Tools (right-click any element > Inspect, or F12), learners click on different elements on the page and observe which CSS rules are highlighted in the Styles panel. The task: for each of the six CSS rules, learners must complete a table with four columns: (1) Selector – what element is being targeted?; (2) Property – what aspect is being changed?; (3) Value – what is it being changed to?; (4) Effect – what does this rule look like on the page? Consolidate by asking learners to write three new CSS rules from scratch and explain each component before applying them. The Developer Tools inspection approach is essential: it shows the live CSS that is actually affecting the page, not just the CSS that is written.

□ **Content Tip:** The anatomy of a CSS rule follows a strict pattern: selector { property: value; }. The selector identifies who to style (e.g., all <p> tags, or elements with class="highlight"). The property identifies what aspect of the element to style (e.g., color, font-size, background-color). The value defines how to style it (e.g., green, 18px, #FF0000). Multiple declarations can be stacked inside one rule: h1 { color: purple; font-size: 32px; font-weight: bold; }. A common beginner error is confusing the colour of text (the color property) with the background colour (background-color). A useful analogy: the selector is "which students in the class?", the property is "which feature of their uniform?", and the value is "what colour is it?"

Learning Activities

Activity 3: Using Different CSS Elements – Implementing the Three Methods

Learning Environment Setup

Artificial (Physical – Classroom):

- Computer lab with individual workstations for learners
- Space for pair discussions and troubleshooting

Technology Environment:

- Computers with text editors (VS Code, Notepad++) and modern browsers
- Projector/board to demonstrate each CSS method live

Natural Environment:

- Interactive environment encouraging experimentation and observation of immediate visual changes
- Collaborative space for learners to compare results and discuss differences

Teaching and Learning Materials

- Computers with browsers and text editor
- Starter HTML file ("Zambia National Park" page)
- Step-by-step instruction guide for Inline, Internal, and External CSS
- Writing materials for notes and observations

Suggested Teaching Methods

- **Demonstration:** Teacher models each CSS method on the projector
- **Hands-on Practice:** Learners implement Inline, Internal, and External CSS on the same page
- **Comparative Analysis:** Learners observe and discuss the advantages and limitations of each method
- **Collaborative Learning:** Pairs troubleshoot and explain each other's results
- **Experiential Learning:** Visual changes reinforce the cascade and specificity rules in CSS

▪ How to Conduct:

Learners work on a single HTML file for a fictional "Zambia National Park" page containing a heading, two paragraphs, and an image. The activity has three sequential stages, each building on the last. Stage 1 – Inline: Learners add a style attribute directly to the <h1> tag to make it green. Discuss the immediate visual result and the limitation: "What if there are 20 headings?" Stage 2 – Internal: Learners move the heading style into a <style> block in the <head>. Add a second rule for paragraphs. Discuss: "Better for one page, but what if the site has 10 pages?" Stage 3 – External: Learners cut all styles from the <style> block, create a new file called styles.css, paste the styles there, and add a <link> tag to connect them. Then create a second HTML page and link the same stylesheet. Change one value in styles.css and observe that both pages update instantly. This three-stage progression makes the value of each method experiential rather than abstract.

□ **Content Tip:** The cascade in 'Cascading Style Sheets' refers to the order in which conflicting CSS rules are resolved. When two rules target the same element, the browser applies a specificity hierarchy to decide which wins: Inline styles (highest specificity, override everything) beat Internal styles, which beat External styles. Within the same level, the rule written last in the file wins. This is why adding a `style=""` attribute to a single element will always override any class or element selector in a stylesheet. For professional web development, the recommended approach is always to use external stylesheets and avoid inline styles entirely. Inline styles make maintenance difficult because the styling is scattered through the HTML rather than centralised in one .css file. Teach this priority order early to prevent confusion when learners later find that their stylesheet rules are being 'overridden'.

Learning Activities

Activity 4: Using Text Properties to Style Fonts

Learning Environment Setup

Artificial (Physical – Classroom):

- Computer lab arranged for individual workstations
- Space for learners to share screens and discuss design choices in pairs

Technology Environment:

- Computers with modern web browsers and text editors
- Projector/board to demonstrate text property application live
- Internet access to preview Google Fonts

Natural Environment:

- Interactive and creative space encouraging experimentation with typography and visual design
- Peer-review area for learners to evaluate readability and aesthetic choices

Teaching and Learning Materials

- Unstyled "Zambia Tourism" HTML page
- Text property reference cards (color, font-family, font-size, font-weight, text-align)
- Access to fonts.google.com for the Google Fonts extension
- Computers with browsers and text editors
- Writing materials for notes and observations

Suggested Teaching Methods

- **Demonstration:** Teacher models each text property and shows live changes
- **Hands-on Practice:** Learners systematically apply color, font-family, font-size, font-weight, and text-align
- **Experiential Learning:** Learners explore Google Fonts and implement imported fonts
- **Peer Review:** Learners evaluate each other's design for readability and aesthetic quality
- **Problem-Based Learning:** Final design challenge to create a professional-looking tourism page
- **Collaborative Learning:** Pairs discuss and justify font choices and styling decisions

How to Conduct:

Provide a "Zambia Tourism" page with unstyled headings and paragraphs. Learners systematically apply text properties in a structured sequence: first color (using a named colour, then a hex code), then font-family (applying a web-safe font like Georgia for headings and Arial for body text), then font-size (using px, then em units), then font-weight (bold vs. normal), then text-align (centre headings, justify paragraphs). After each property, learners note the change and save their work. Introduce Google Fonts: show learners how to copy a `<link>` import code, add it to the `<head>`, and use the font in CSS (font-family: 'Roboto', sans-serif). Conclude with a design challenge: "Style the page so it looks like a professional tourism brochure. Use at least two different fonts – one for headings and one for body text." Learners peer-review and evaluate readability.

□ **Content Tip:** Typography is one of the most powerful tools in web design, but beginners often underestimate it. Font pairing (using two complementary fonts – typically a decorative or serif font for headings and a clean sans-serif font for body text) significantly improves readability and visual hierarchy. Google Fonts (fonts.google.com) provides over 1,000 free, web-optimised fonts that can be added with a single <link> tag – a huge upgrade from the limited set of web-safe fonts. When using the color property, named colours (red, blue, green) are limited to about 140 options. Hexadecimal codes (#RRGGBB) provide access to over 16 million colours and are the professional standard. The browser Developer Tools are invaluable for experimenting: click on any element, find the color property in the Styles panel, and click the colour swatch to open a live colour picker that updates the page instantly.

Learning Activities

Activity 5: Managing the Box Model

Learning Environment Setup

Artificial (Physical – Classroom):

- Computer lab arranged for individual and pair workstations
- Space at the board for visual demonstration using the picture frame analogy

Technology Environment:

- Computers with modern web browsers and text editors
- Browser Developer Tools (Inspect Element) to demonstrate live box model changes
- Projector/board to show diagram examples and CSS changes

Natural Environment:

- Interactive and exploratory environment encouraging learners to test layouts and observe immediate effects, Peer collaboration area for comparing layout solutions

Teaching and Learning Materials

- Computers with browsers and text editors (VS Code, Notepad++, Chrome/Firefox Developer Tools)
- Physical picture frame analogy diagram (printed for reference)
- Box model CSS practice worksheet with a three-image layout challenge
- Starter CSS file including box-sizing: border-box

Suggested Teaching Methods

- **Demonstration:** Teacher explains box model using physical analogy and Developer Tools
- **Hands-on Practice:** Learners modify margin, padding, and border values in live CSS
- **Experiential Learning:** Students apply box model properties to a practical layout challenge (three images in a row)
- **Peer Review:** Learners compare results and discuss differences in spacing and alignment
- **Guided Discovery:** Students explore how padding, margin, and border affect layout and total element size
- **Problem-Based Learning:** Challenge learners to create balanced, visually appealing layouts using only box model properties

▪ How to Conduct:

Begin with a visual demonstration using a physical analogy: draw a picture frame on the board. The picture itself is the content. The mat inside the frame between the picture and the frame border is the padding. The frame itself is the border. The space between the frame and the wall (or other frames) is the margin. Map each physical component to its CSS equivalent. Then demonstrate the Browser Developer Tools box model panel: right-click any element > Inspect > click the element in the Elements panel > scroll to the bottom of the Styles panel to see the live box model diagram (a nested rectangle showing exact pixel values for margin, border, padding, and content). Learners then modify these values using CSS rules and observe the live effects. Set a practical challenge: layout three images in a row with equal spacing between them using only margin and padding – not tables.

□ **Content Tip:** The box model is the foundational layout concept of CSS and the source of many beginner layout frustrations. The critical distinction: padding is space inside the border (between the content and its border) – it takes the element’s background colour. Margin is space outside the border (between the element and its neighbours) – it is always transparent. Common pitfalls: (1) Adding too much margin causes mysterious gaps between elements. (2) Adding too much padding makes a container appear larger than intended. (3) Borders add to the total size of an element by default (this can be changed with box-sizing: border-box, a property worth introducing). The box-sizing: border-box rule is used by nearly every professional CSS framework (Bootstrap, Tailwind) because it makes size calculations far more predictable – include it in your starter stylesheet.

Learning Activities

Activity 6: Applying Background Properties

Learning Environment Setup

Artificial (Physical – Classroom):

- Computer lab arranged for individual or paired learners

Technology Environment:

- Computers with modern browsers and text editors, Access to online image libraries (for background images)
- Projector/board to show live HTML pages with background changes

Natural Environment:

- Interactive, exploratory setting where learners test visual effects of colors and images

Teaching and Learning Materials

- Computers with browsers and text editors (VS Code, Notepad++, Chrome/Firefox Developer Tools)
- Starter HTML page (“Zambia Tourism” page) and Sample images for background demonstration
- Background properties reference card (background-color, background-image, background-repeat, background-position, background-size, background-attachment)
- Local landmark image files or royalty-free image URLs
- Worksheet with practical exercises for applying background properties

Suggested Teaching Methods

- **Demonstration:** Teacher shows live effects of different background colors and images
- **Hands-on Practice:** Learners apply background properties on the starter HTML page
- **Guided Exploration:** Learners test combinations of color, image, repeat, position, and size
- **Problem-Based Learning:** Assign challenge to create a visually appealing tourism page using proper background styling
- **Peer Review:** Students compare pages and provide feedback on readability and aesthetics
- **Experiential Learning:** Encourage learners to observe how changes in background properties affect the overall layout and user experience

▪ How to Conduct:

Start by showing three visual examples: (1) a solid background colour; (2) a full-screen background image; (3) a repeating pattern background image. For each, learners examine the CSS rule that produced it and identify the properties used. Learners then apply each type to sections of their Tourism Zambia page: set a background-color for the header (using a dark Zambian-themed colour), apply a background-image to the hero section (using a local landmark image URL or a downloaded file), and set it to no-repeat, center, cover so the image fills the section regardless of screen size. Introduce background-position (placing the image focal point) and the shorthand background property that combines all sub-properties in one line. Extension: learners add a background-color as a fallback (for when the image fails to load) and verify its function by temporarily disabling the image in Developer Tools.

□ **Content Tip:** Background properties are among the most versatile in CSS. The most important values to master: background-color provides a solid colour fill and acts as a fallback if an image fails to load. background-image: url('...') applies an image, which repeats by default (background-repeat: repeat). For a large hero/banner image that should fill the entire section, the combination background-size: cover; background-position: center; background-repeat: no-repeat; is the standard professional pattern. The shorthand background property combines all sub-properties in one line: background: #3A3A3A url('hero.jpg') no-repeat center center / cover; The order matters and can be confusing – provide learners with this exact pattern as a template and explain each component, rather than expecting them to memorise the order from scratch.

Learning Activities

Activity 7: Evaluating Web Accessibility

Learning Environment Setup

Artificial (Physical – Classroom):

- Computer lab with individual or paired workstations
- Space for teacher demonstration and group discussion

Technology Environment:

- Computers with modern browsers (Chrome or Firefox) and Developer Tools
- Browser accessibility extensions (e.g., Spectrum for simulating colour blindness)
- Internet access for WebAIM Contrast Checker
- Projector/board to demonstrate live accessibility checks

Natural Environment:

- Empathy-driven, inclusive environment encouraging learners to consider diverse user needs
- Collaborative atmosphere for sharing observations and solutions

Teaching and Learning Materials

- Computers with browsers and Developer Tools
- Access to WebAIM Contrast Checker (webaim.org/resources/contrast-checker)
- Colour blindness simulation tool (Spectrum or similar extension)
- Starter HTML/CSS page designed by learners for accessibility testing
- Worksheet to record observations, contrast ratio results, and responsive design issues

Suggested Teaching Methods

- **Demonstration:** Teacher shows how to use DevTools and extensions to test accessibility
- **Hands-on Practice:** Learners test their own pages for colour contrast, readability, and mobile responsiveness
- **Guided Exploration:** Learners adjust colours, font sizes, and layout to meet WCAG standards
- **Empathy-Based Learning:** Simulate disabilities to understand real user experience
- **Problem-Based Learning:** Challenge learners to redesign any failing elements for accessibility compliance
- **Peer Review:** Students evaluate each other's pages for inclusivity and mobile usability

How to Conduct:

Open with an empathy exercise: using the Accessibility settings in Chrome DevTools (or a browser extension like "Spectrum"), simulate how the page appears to users with different types of colour blindness (deuteranopia, protanopia). Learners observe whether their designed page is still readable and navigable. Introduce the WCAG (Web Content Accessibility Guidelines) contrast ratio standard: text must have a contrast ratio of at least 4.5:1 against its background. Learners use WebAIM's Contrast Checker (webaim.org/resources/contrastchecker) to test their colour combinations and adjust those that fail. Extend to responsive design: learners resize their browser window from full-screen to a narrow mobile width and observe where the layout "breaks" (text overflows, images are cropped, elements overlap). Discuss why this matters for learners in Zambia, where most internet access is via mobile phone.

□ **Content Tip:** Web accessibility is an ethical imperative, not an optional feature. In Zambia, an estimated 1.4 million people live with some form of disability (WHO data), and globally, approximately 8% of males have some form of colour vision deficiency. A website that is only usable by people with perfect vision and the latest laptop immediately excludes a significant portion of the population. The WCAG 4.5:1 contrast ratio for normal text is the minimum legal standard in many countries and the professional baseline for all web projects. Additionally, in Zambia as elsewhere in Africa, the majority of internet users access the web via smartphone on a mobile data connection – designing only for a desktop screen in a computer lab creates pages that are unusable in the most common real-world context. Responsive design (using percentage widths, max-width, and CSS media queries) is therefore a practical and inclusive necessity.

Expected Standard

CSS concepts are applied to websites correctly.

ASSESSMENT

Examples of Formative

Scenario: You are helping a family member build a simple website for their restaurant in Kitwe called “Mama B’s Kitchen.” You want all page titles (<h1> elements) to be dark green, and the main text (<p>) to be dark grey on a light background.

1. (Applying Style Methods) What is the most professional and efficient way to apply these styles? Why is this method better than the others, especially if more pages are added later?
2. (Box Model) You try to put a border around the restaurant’s logo, but the text on the next line is touching the border. Which part of the CSS box model (margin or padding) would you increase to fix this?

Formative Rubric:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Understanding CSS Role	Clearly explains how CSS separates content from presentation and why it is important.	Can state that CSS styles HTML.	Has a vague idea but confuses CSS with HTML.	Does not understand the role of CSS.
Applying Style Methods	Correctly implements all three methods (inline, internal, external) and explains when to use each.	Implements at least two methods correctly with guidance.	Attempts to apply styles but makes syntax errors.	Cannot apply any CSS method.
Using Selectors	Skilfully uses element, class, and ID selectors to target specific elements.	Uses element selectors correctly; attempts class or ID.	Only uses basic selectors with errors.	No understanding of selectors.
Box Model Manipulation	Independently adjusts margin, padding, and border to achieve precise layouts.	With guidance, changes margin and padding to alter spacing.	Attempts changes but cannot predict the effect.	No attempt or cannot modify box model properties.
Accessibility Awareness	Evaluates and improves a page’s contrast/responsiveness for accessibility.	Discusses the need for contrast and responsive design.	Aware of terms but cannot apply them.	No awareness of accessibility issues.

Example of Summative Assessment:

Scenario: A local tour company in Livingstone, “Victoria Falls Adventures”, wants a simple one-page website. They have given you the HTML structure. Your job is to add the CSS to make it look professional and inviting.

Task – Design Project:

Create an external CSS file called style.css and link it to their HTML page. Your CSS must:

1. Use an element selector to set the main font for the whole page and a light, earthy background colour.
2. Use a class selector (e.g., .tour-price) to make all tour prices bold and dark green.
3. Use the box model (padding and margin) to add space around the main description text.
4. Ensure all text has good contrast against the background for readability by all visitors.

SUMMATIVE RUBRIC:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
External CSS Implementation	External CSS is well-organised, commented, and linked correctly across all three pages.	External CSS is correctly linked; styles applied consistently.	CSS is partially external; some inline styles remain or linking errors occur.	No external CSS; styles missing or incorrect.
Styling Consistency	All pages have a consistent, professional look with coherent colour scheme and typography.	Pages have a consistent style (same fonts, colours).	Some inconsistency in styling between pages.	Pages look different or lack styling.
Box Model & Layout	Layout is well-structured, with effective use of margin, padding, and border for alignment.	Layout is readable, with appropriate spacing.	Layout is cluttered or elements overlap due to box model misuse.	No apparent use of box model; layout chaotic.
Responsive Design	Site adapts smoothly to mobile, tablet, and desktop views; no broken elements.	Site is mostly usable on different screen sizes.	Site has issues on smaller screens (e.g., horizontal scroll).	Site not responsive; unusable on mobile.
Content & Creativity	Content is engaging, relevant, and well-written; design enhances the business theme.	Content is appropriate and complete.	Content is minimal or generic.	Content missing or not related to business.

Summary – Key Points Recap:

- CSS separates content (HTML) from visual design.
- A CSS rule uses a selector, property, and value to style elements.
- External stylesheets allow site-wide changes from one file, making them efficient for multi-page sites.
- The box model (content, padding, border, margin) controls the layout and spacing of elements.
- Responsive design ensures accessibility and a good user experience across all screen sizes.

TOPIC 2.8: DATABASES

Overview

Databases are organised collections of data that allow for efficient storage, retrieval, and management of information. This topic introduces Structured Query Language (SQL), the universal standard for interacting with relational databases. Learners will move from understanding basic data organisation to designing schemas and writing SQL scripts to create and modify database structures.

General Competence(s):

Analytical Thinking	Learners analyze database requirements, identify relationships, and structure data logically.
Application Software Skills	Using database management systems (DBMS) like MySQL, PostgreSQL, or MS Access to implement designs.
Collaboration and Communication	Working in pairs or groups to design schemas and discuss queries, sharing insights effectively.
Critical Thinking and Problem-Solving	Debugging SQL scripts, resolving data inconsistencies, and optimizing queries.
Data Structure Skills	Understanding tables, fields, keys, and relationships to organize and normalize data efficiently.
Digital Literacy	Navigating DBMS interfaces, writing and testing SQL queries, and interpreting results.
Programming Skills	Writing SQL scripts to create, modify, and manipulate databases programmatically.
Technical Proficiency	Applying technical knowledge to implement and manage database structures and queries successfully.

Hook

"How does a supermarket in Lusaka keep track of thousands of items and know exactly when to order more sugar?" Follow up: "If they used one giant spreadsheet for everything, what problems might they run into?" – leading to the idea of organised tables.

Key Terms / Words / Vocabulary

SQL (Structured Query Language): A standard programming language used to manage and manipulate data in relational databases.

Relational Database: A type of database that organises data into one or more tables that can be linked to each other based on common data.

Table: A collection of related data organised into rows and columns within a database.

Field / Column: A single piece of information within a table, representing a specific attribute (e.g., "FirstName").

Record / Row: A complete set of information about a single item or entry in a table.

Primary Key: A unique identifier for each record in a database table. Cannot be empty and must be unique for every row.

Schema: The structure or blueprint of a database, defining its tables, fields, relationships, and rules.

Normalisation: The process of organising data in a database to reduce redundancy and improve data integrity.

Sub-Topic 2.8.1: Database Design and Creation

Introduction: This sub-topic introduces database design and creation, focusing on how to organise and manage data effectively. Teachers should guide learners to design database schemas that define tables, fields, and relationships. Learners should practise writing SQL scripts to create and modify database structures, and learn how to query and manipulate data using SQL commands. Emphasise hands-on activities to help learners understand how databases store, retrieve, and manage information efficiently.

Specific Competence(s) – Learners will:

- 2.8.1.1 Design and create databases using SQL.

Learning Activities

Activity 1: Discussing Structured Query Language (SQL) Concepts

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for group work and pair discussions
- Space to lay out index cards for the physical database simulation

Technology Environment:

- Computers with internet access to run SQL in an online sandbox (SQLiteOnline.com or W3Schools SQL Tryit)
- Projector/whiteboard for demonstrating SQL commands live

Natural Environment:

- Interactive and collaborative environment encouraging discussion and exploration
- Hands-on activities to link abstract concepts to real-world contexts

Teaching and Learning Materials

- Index cards for physical database simulation (customers and orders)
- Computers with internet access for SQL practice
- Pre-created sample Customers database (SQL file to import)
- Printed SQL command cheat sheet (SELECT, WHERE, INSERT, UPDATE)
- Whiteboard/markers to explain SQL concepts and map table structures

Suggested Teaching Methods

- **Demonstration:** Teacher shows SQL commands in the sandbox environment
- **Experiential Learning:** Physical card simulation to illustrate database queries
- **Guided Practice:** Learners run SELECT, INSERT, and UPDATE commands on sample tables
- **Collaborative Learning:** Pairs or small groups explore queries and share findings
- **Analogy-Based Teaching:** Compare database tables to spreadsheets for easier conceptual understanding

▪ How to Conduct:

Use a physical simulation to introduce the concept before touching a computer. Create a physical "database" using index cards: a stack of "customer" cards (each with a name, phone number, and town) and a stack of "order" cards (each with an order number, item, and a customer name written on it). Ask learners to find all orders for a customer named "Chanda" – they physically search through the cards. Now ask: "If there were 10,000 cards, how long would that take?" Introduce SQL as the language that allows us to ask a database this exact question instantly: `SELECT * FROM Orders WHERE CustomerName = 'Chanda'`. Move to a computer with an online SQL sandbox (SQLiteOnline.com or W3Schools SQL Tryit). Using a pre-populated Customers table, guide learners to run: `SELECT *` (all columns), `SELECT` specific columns, and `SELECT WHERE` to filter. Build progressively to `INSERT` (add a new customer) and `UPDATE` (change a customer's phone number).

□ **Content Tip:** SQL is the universal language for relational databases – it works on MySQL, PostgreSQL, Microsoft SQL Server, SQLite, and Oracle with minor differences in syntax. Frame SQL as a conversation with the database: SELECT is asking a question (“Show me...”), INSERT is adding information (“Record this...”), UPDATE is correcting information (“Change this...”), and DELETE is removing information (“Remove this...”). The asterisk (*) in SELECT * means “all columns” – a convenient shortcut but a bad practice in production systems where tables have many columns, as it retrieves more data than needed. Teach learners to be specific: SELECT FirstName, LastName FROM Customers is always better practice than SELECT * FROM Customers in professional contexts. Connect to the spreadsheet analogy: a database table is like a spreadsheet sheet, a row is a record, and a column is a field – but a database can link multiple tables, something a spreadsheet cannot do natively.

Learning Activities

Activity 2: Designing Database Schemas

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for group work and collaborative ERD design
- Space to lay out index cards for demonstrating data redundancy and relationships
- Board/flip chart used to model table structures, keys, and ERD diagrams step-by-step

Technology Environment:

- Index cards for physical database simulation (customers and orders)
- Computers with internet access for designing ERDs using tools like draw.io or Lucidchart
- Projector/whiteboard for demonstrating how to convert physical tables into ERDs
- Optional: database software (e.g., MS Access/MySQL) for extended practice

Natural Environment:

- Interactive and collaborative environment encouraging discussion on real-life data problems
- Use of familiar contexts such as a school library system to connect concepts to real-world applications
- Hands-on activities linking physical simulation to abstract database design concepts

Teaching and Learning Materials

- Index cards for demonstrating redundancy (customer/order or book/borrowing records)
- ERD design templates (rectangles for entities and lines for relationships)
- Chart paper, pencils, rulers for drawing ERDs,
- Sticky notes for modifying table structures during design
- Computers/tablets for digital ERD creation
- Sample scenario: School library database (Books and BorrowingRecords tables)
- Whiteboard/markers for illustrating normalisation, primary keys, and foreign keys
- ERD drawing tools: paper/pencils, or online tools like draw.io or Lucidchart

Suggested Teaching Methods

- **Demonstration:** Teacher models redundancy problem and introduces CustomerID/BookID as primary keys
- **Experiential Learning:** Learners use index cards to physically simulate tables and relationships
- **Guided Discovery:** Learners identify problems of repeated data and derive the need for normalization
- **Collaborative Learning:** Small groups design ERDs for the library system and present their diagrams

- **Problem-Based Learning:** Learners solve the issue of data inconsistency by restructuring tables
- **Visual Learning:** Use of ERDs to represent entities, attributes, and relationships clearly
- **Scaffolding:** Progression from physical simulation → table structure → ERD diagram design

▪ How to Conduct:

Return to the physical simulation: learners observed that the customer's name was written on both the customer card and the order card. Ask: "What problem does this cause?" (If Chanda gets married and changes her name, we have to update it on every single order card – risking errors and inconsistency.) Introduce this as the core problem that relational database design solves: data normalisation. Explain the solution: give each customer a unique ID number (CustomerID). The order card now records only the CustomerID, not the full name. To get the customer's name, the database "joins" the two tables using the matching ID. Learners then design an Entity-Relationship Diagram (ERD) for a school library system: a Books table (BookID, Title, Author, Year) and a BorrowingRecords table (RecordID, BookID, StudentID, DateBorrowed, DueDate). Use index cards physically to show the relationship before drawing the formal ERD.

□ **Content Tip:** Normalisation is the discipline of organising data to eliminate redundancy and ensure data integrity. The core principle at this level is First Normal Form (1NF): each cell in a table should contain one piece of information, and no data should be unnecessarily repeated across tables. The Primary Key is the cornerstone of a well-designed table: it uniquely identifies each record and cannot be NULL or duplicated. A Foreign Key in one table is a Primary Key from another table – it is the link that creates the relationship between tables. The ERD (Entity-Relationship Diagram) is the standard tool for planning a database before writing any SQL. It shows each table as a rectangle with its field names listed, and lines between tables showing the relationships (one-to-many, many-to-many). Sketching an ERD on paper before writing code is the database equivalent of designing a flowchart before programming – an investment that saves significant debugging time.

Learning Activities

Activity 3: Writing SQL Scripts to Create and Modify Structures

Learning Environment Setup

Artificial (Physical – Classroom):

- ERD drawing tools: paper/pencils, or online tools like draw.io or Lucidchart
- Classroom arranged for individual work and pair programming
- Board/whiteboard used to break down SQL syntax step-by-step
- Space for learners to refer to their ERDs while writing SQL scripts

Technology Environment:

- Computers with SQL environments such as MySQL Workbench, MariaDB, or SQLiteOnline
- Internet access for executing and testing SQL scripts
- Projector/whiteboard for live demonstration of SQL commands

Natural Environment:

- Interactive and exploratory environment encouraging experimentation and debugging
- Real-life context (library system) used to connect SQL scripts to practical applications
- Hands-on activities linking ERD design to actual database implementation

Teaching and Learning Materials

- Computers with MySQL Workbench, MariaDB, or SQLite Online or online SQL platforms
- Step-by-step SQL script writing guide
- Data types reference card (INT, VARCHAR, DATE, DECIMAL, NOT NULL, AUTO_INCREMENT)
- ERD from Activity 2 (Books and Borrowing Records tables)
- Whiteboard and markers for explaining SQL syntax
- Sample SQL scripts for demonstration and practice

Suggested Teaching Methods

- **Demonstration:** Teacher writes and explains SQL scripts line-by-line
- **Guided Practice:** Learners follow instructions to create and verify tables
- **Experiential Learning:** Learners execute SQL commands and observe system responses
- **Collaborative Learning:** Learners work in pairs to debug and correct SQL errors
- **Scaffolding:** Progression from CREATE TABLE → FOREIGN KEY → ALTER TABLE
- **Problem-Based Learning:** Learners solve a practical task (adding ISBN field to a table)

▪ How to Conduct:

Guide learners to translate the ERD they designed in Activity 2 into executable SQL. Begin with the CREATE TABLE script for the Books table, explaining each component live: CREATE TABLE Books (– opens the table definition; BookID INT PRIMARY KEY AUTO_INCREMENT, – creates a unique integer ID that increases automatically; Title VARCHAR(100) NOT NULL, – creates a text field, maximum 100 characters, cannot be empty; Author VARCHAR(100), – creates a text field, which can be empty (nullable); Year INT – creates an integer field for the publication year;); – closes the definition. Learners run the script in their SQL environment and verify the table was created by running SELECT * FROM Books (which returns an empty table – this is correct!). Then add the BorrowingRecords table, including a FOREIGN KEY constraint linking BookID to the Books table. Close with a practical extension: the librarian wants to add an ISBN number field. Learners write ALTER TABLE Books ADD COLUMN ISBN VARCHAR(20);

□ **Content Tip:** Data types in SQL are critical for ensuring data integrity and efficient storage. The most common types at this level: INT (whole numbers – for IDs, quantities, years), VARCHAR(n) (variable-length text up to n characters – for names, descriptions, codes), DATE (a calendar date in YYYY-MM-DD format – for due dates, birth dates), and DECIMAL(p,s) (precise decimal numbers with p total digits and s decimal places – for prices and amounts). The NOT NULL constraint prevents a field from being left empty – essential for fields that must always have a value (like a primary key or a person's name). AUTO_INCREMENT (MySQL) or AUTOINCREMENT (SQLite) automatically assigns the next available integer as the primary key when a new record is inserted, eliminating the need to manually specify ID numbers. Using consistent naming conventions (CamelCase for table names: Books, BorrowingRecords; lowercase_underscores for field names: book_id, due_date) makes SQL scripts far more readable and professional.

Learning Activities

Activity 4: Querying and Manipulating Data with SQL

Learning Environment Setup

Artificial (Physical – Classroom):

- Classroom arranged for individual practice and pair discussions
- Board/whiteboard used to demonstrate SQL queries step-by-step
- Space for learners to record and compare query results (query logs)

Technology Environment:

- Computers with SQL database environments such as MySQL Workbench, MariaDB, or SQLiteOnline
- Pre-populated school library database (Books and Borrowing-Records tables)
- Projector for live demonstration of SQL queries and outputs

Natural Environment:

- Interactive environment encouraging exploration, testing, and debugging of queries
- Real-life context (library system) to connect SQL queries to meaningful information retrieval
- Hands-on practice linking database operations to real-world decision-making (e.g., overdue books, popular books)

Teaching and Learning Materials

- Sample SQL scripts for demonstration and practice
- Computers with SQL database environment (pre-populated library database)
- SQL query progression worksheet (simple to complex queries)
- Query log template for recording queries and their outputs
- SQL operator reference card (=, >, <, LIKE, BETWEEN, ORDER BY, etc.)
- Whiteboard and markers for explaining query logic, Sample SQL queries for demonstration

Suggested Teaching Methods

- **Demonstration:** Teacher models SQL queries (SELECT, WHERE, ORDER BY, INSERT, UPDATE) and explains outputs
- **Guided Practice:** Learners follow structured tasks from simple retrieval to complex queries
- **Experiential Learning:** Learners execute queries and observe real-time results from the database
- **Collaborative Learning:** Learners work in pairs to test queries and compare results
- **Scaffolding:** Progression from simple queries → filtering → sorting → data manipulation
- **Problem-Based Learning:** Learners solve real tasks such as identifying overdue books and most borrowed books
- **Inquiry-Based Learning:** Learners create their own SQL queries to answer analytical questions

▪ How to Conduct:

Pre-populate the school library database with 20 book records and 15 borrowing records. Learners practice a progression of queries from simple to complex. Simple retrieval: `SELECT * FROM Books;` and `SELECT Title, Author FROM Books;`. Filtering: `SELECT * FROM Books WHERE Year > 2010;` and `SELECT * FROM BorrowingRecords WHERE DueDate < '2026-01-01';` (overdue books). Sorting: `SELECT * FROM Books ORDER BY Author ASC;` and `SELECT * FROM Books ORDER BY Year DESC;`. Inserting: `INSERT INTO Books (Title, Author, Year) VALUES ('The River Between', 'Ngũgĩ wa Thiong'o', 1965);`. Updating: `UPDATE Books SET Year = 1966 WHERE BookID = 3;`. Learners document each query and its result in a query log table. Conclude with an open-ended analytical task: "Write three SQL queries that would give the librarian useful information about which books are most borrowed and which have overdue returns."

□ **Content Tip:** The WHERE clause is the most powerful tool for data retrieval because it filters the result set to show only the records that are relevant. Common operators in WHERE clauses: = (equals), != or <> (not equals), > and < (greater than, less than), >= and <= (greater than or equal to, less than or equal to), LIKE '%pattern%' (finds records containing a text pattern – e.g., WHERE Title LIKE '%Zambia%' finds all books with 'Zambia' anywhere in the title), and BETWEEN x AND y (finds values in a range). The ORDER BY clause organises the output: ASC sorts ascending (A to Z, lowest to highest), DESC sorts descending (Z to A, highest to lowest). A practical tip: always use WHERE carefully with UPDATE and DELETE statements – omitting WHERE updates or deletes every row in the table, which is almost never the intended action. The habit of writing SELECT * FROM TableName WHERE condition first to preview which rows will be affected, before changing SELECT * to UPDATE or DELETE, is a professional safeguard.

Expected Standard

Databases designed and created using SQL correctly.

ASSESSMENT

Examples of Formative

Scenario: A small video club in Matero, Lusaka, wants to move from a paper list to a computer database. They have one notebook where they write the customer's name, the movie they rented, and the date. They notice that if "Banda" rents a movie three times, his name is written over and over again.

1. (Schema Design) Why is this "one notebook" method inefficient? Draw a quick sketch showing how you could split this information into two tables (Customers and Rentals) to avoid repeating the customer's name every time. Show a simple link between them using a CustomerID.
2. (SQL Syntax) If the Movies table already exists with columns for MovieID and Title, write the SQL command to add a new column called ReleaseYear to this table.

Formative Rubric:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Schema Design (ERD)	Designs a normalised schema with multiple related tables, correctly identifying primary keys.	Designs a single table with appropriate fields and a primary key.	Table design missing key fields or primary key.	No design or completely off-topic.
SQL Syntax (CREATE)	Writes complex CREATE TABLE scripts with data types and constraints correctly.	Writes a basic CREATE TABLE script that executes without errors.	Script has syntax errors that prevent execution.	Cannot write a CREATE TABLE statement.

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
SQL Syntax (ALTER)	Successfully modifies table structure using ALTER with multiple changes.	Uses ALTER to add a single field correctly.	Attempts ALTER but makes syntax errors.	No attempt or unable to use ALTER.
Using ORDER BY	Uses ORDER BY with multiple columns and directions correctly in queries.	Uses ORDER BY correctly to sort results.	Attempts ORDER BY but syntax is wrong.	Cannot use ORDER BY.

Example of Summative Assessment:

Scenario (Practical Test): A local football club in Mufulira, the “Mufulira Wanderers,” wants a simple database to track its players. The club has a Youth Team (Under-17) and a Senior Team. A player can only be on one team at a time.

Task:

1. Design: Draw the schema for two tables: Teams and Players. Teams should have TeamID (Primary Key) and TeamName. Players should have PlayerID (Primary Key), PlayerName, and a TeamID field (Foreign Key linking to the Teams table).
2. SQL Scripts: Write the SQL CREATE TABLE script for the Players table. Include correct data types (INT for IDs, VARCHAR for names) and the Primary Key.
3. Data Query: The coach wants to see all players sorted alphabetically. Write the SQL SELECT statement to do this (e.g., `SELECT * FROM Players ORDER BY PlayerName ASC;`).

SUMMATIVE RUBRIC:

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
Schema Design	Schema includes multiple related tables with appropriate primary keys and relationships; fully normalised.	Schema includes all required tables with correct fields and primary keys.	Schema is incomplete or missing primary keys.	No schema or unrelated to scenario.
CREATE TABLE Scripts	All scripts run without errors and create the exact tables as designed.	Scripts run with minor errors that are quickly fixed; tables created correctly.	Scripts have significant errors; tables not created properly.	Scripts do not run or not provided.
Data Insertion (if required)	Inserts sample data correctly using INSERT statements.	Inserts some data but may have errors.	Data insertion is incorrect or missing.	No data inserted.

Criteria	Exceeding (4)	Meeting (3)	Approaching (2)	Beginning (1)
ORDER BY Query	Writes a correct and efficient ORDER BY query that answers a specific analytical question.	Writes a correct ORDER BY query to sort data.	Query has syntax errors or does not sort as intended.	Cannot write a SELECT with ORDER BY.
Documentation	Provides clear comments and explanation of the database design and SQL commands.	Provides basic documentation of tables and fields.	Documentation is sparse or unclear.	No documentation provided.

Summary – Key Points Recap:

- A database is an organised collection of data, managed by a DBMS.
- SQL is the standard language for managing and manipulating relational databases.
- Key SQL commands include CREATE TABLE, INSERT, SELECT, UPDATE, ALTER, and ORDER BY.
- Good database design involves planning a schema with defined tables, fields, and a primary key to ensure data integrity.
- Normalisation helps reduce data redundancy, and security measures like encryption protect sensitive information.

REFERENCES

Spreadsheets and Productivity Tools

1. Alexander, M., & Kusleika, D. (2022). *Excel 2021 / Microsoft 365 bible*. John Wiley & Sons.
2. Frye, C. (2019). *Microsoft Excel 2019 step by step*. Microsoft Press.
3. Harvey, G. (2019). *Excel 2019 all-in-one for dummies*. John Wiley & Sons.
4. Jelen, B. (2022). *Excel 2021 in easy steps*. In Easy Steps Limited.
5. McFedries, P. (2020). *Excel 2019 formulas and functions*. Microsoft Press.
6. Walkenbach, J. (2019). *Excel 2019 power programming with VBA*. John Wiley & Sons.

Programming and Algorithms

7. Briggs, J. R. (2019). *Python for kids: A playful introduction to programming* (2nd ed.). No Starch Press.
8. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
9. Marji, M. (2014). *Learn to program with Scratch: A visual introduction to programming with games, art, science, and math*. No Starch Press.
10. Sweigart, A. (2020). *Automate the boring stuff with Python: Practical programming for total beginners* (2nd ed.). No Starch Press.
11. Woodcock, J. (2016). *Coding games in Scratch*. DK Children.

Web Development

12. Flanagan, D. (2020). *JavaScript: The definitive guide* (7th ed.). O'Reilly Media.
13. Freeman, E., & Robson, E. (2020). *Head first HTML and CSS* (2nd ed.). O'Reilly Media.
14. McFarland, D. S. (2020). *CSS: The missing manual* (4th ed.). O'Reilly Media.
15. Meyer, E. A., & Weyl, E. (2023). *CSS: The definitive guide* (5th ed.). O'Reilly Media.
16. Robbins, J. N. (2018). *Learning web design: A beginner's guide to HTML, CSS, JavaScript, and web graphics* (5th ed.). O'Reilly Media.

Database Systems

17. Beaulieu, A. (2020). *Learning SQL: Generate, manipulate, and retrieve data* (3rd ed.). O'Reilly Media.
18. Churcher, C. (2016). *Beginning SQL queries: From novice to professional* (2nd ed.). Apress.
19. Forta, B. (2022). *Sams teach yourself SQL in 10 minutes* (6th ed.). Sams Publishing.
20. Rockoff, L. (2022). *The language of SQL* (3rd ed.). Addison-Wesley Professional.

21. Viescas, J. L. (2018). *SQL queries for mere mortals* (4th ed.). Addison-Wesley Professional.

Cybersecurity

22. Baase, S. (2018). *A gift of fire: Social, legal, and ethical issues for computing technology* (5th ed.). Pearson Education.
23. Ciampa, M. (2020). *CompTIA Security+ guide to network security fundamentals* (7th ed.). Cengage Learning.
24. Easttom, C. (2019). *Computer security fundamentals* (4th ed.). Pearson Education.
25. Grimes, R. A. (2017). *Hacking the hacker: Learn from the experts who take down hackers*. John Wiley & Sons.
26. Hadnagy, C. (2018). *Social engineering: The science of human hacking* (2nd ed.). John Wiley & Sons.

Data Representation and Number Systems

27. Mano, M. M., & Ciletti, M. D. (2018). *Digital design: With an introduction to the verilog HDL, VHDL, and systemVerilog* (6th ed.). Pearson Education.
28. Tocci, R. J., Widmer, N. S., & Moss, G. L. (2017). *Digital systems: Principles and applications* (12th ed.). Pearson Education.
29. White, S. A. (2019). *Understanding digital electronics*. Elektor International Media.

Networking

30. Lowe, D. (2021). *Networking all-in-one for dummies* (8th ed.). John Wiley & Sons.
31. Odom, W. (2020). **CCNA 200-301 official cert guide library**. Cisco Press.
32. Seifert, R., & Edwards, J. (2020). *The all-new switch book: The complete guide to LAN switching technology* (2nd ed.). Wiley Publishing.

Online Resources and Documentation

33. [Code.org](https://code.org/educate/csf). (2024). *CS fundamentals*. Retrieved from <https://code.org/educate/csf>
34. Google for Education. (2024). *CS First: Computer science curriculum*. Retrieved from <https://csfirst.withgoogle.com>
35. MDN Web Docs. (2024). *CSS: Cascading Style Sheets*. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/CSS>
36. MDN Web Docs. (2024). *HTML: HyperText Markup Language*. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTML>
37. MySQL. (2024). *MySQL documentation*. Retrieved from <https://dev.mysql.com/doc>
38. Python Software Foundation. (2024). *The Python tutorial*. Retrieved from <https://docs.python.org/3/tutorial>

- 39. Scratch Foundation. (2024). *Scratch educator guide*. Retrieved from <https://scratch.mit.edu/educators>
- 40. SQLite. (2024). *SQLite documentation*. Retrieved from <https://www.sqlite.org/docs.html>
- 41. W3C. (2024). *Web Accessibility Initiative (WAI)*. Retrieved from <https://www.w3.org/WAI>
- 42. W3Schools. (2024). *CSS tutorial*. Retrieved from <https://www.w3schools.com/css>
- 43. W3Schools. (2024). *SQL tutorial*. Retrieved from <https://www.w3schools.com/sql>

Zambian Educational Resources

- 44. Curriculum Development Centre. (2023). Computer Science teaching syllabus: Forms 1-4. Ministry of Education, Republic of Zambia.
- 45. Curriculum Development Centre. (2023). *Zambia Education curriculum framework*. Ministry of Education, Republic of Zambia.
- 46. Zambia Information and Communications Technology Authority. (2023). *Cyber security awareness guide*. Retrieved from <https://www.zicta.zm>

APPENDIX: LESSON PLAN

MINISTRY OF EDUCATION LESSON PLAN

SCHOOL: WCE TECH GIRLS

NAME OF TEACHER: Sakandyendye

DATE: 24/03/2026

CLASS/ LEVEL: FORM 2

DURATION: 80 Minutes (2 PERIODS)

TIME: 08:20 - 09:40

NO. OF PUPILS: 54 BOYS: 25 GIRLS: 29

SUBJECT: Computer Science

TOPIC: 2.4 Networking (This must come from the syllabus with its serial number)

SUB-TOPIC: 2.4.1 Network Topologies (*This must come from the syllabus with its serial number*)

GENERAL COMPETENCES:

Technical Proficiency, Networking Knowledge, Communication, Collaboration, Critical Thinking and Problem-Solving

SPECIFIC COMPETENCES: (*This must come from the syllabus with its serial number*)

2.4.1.1 Develop and Configure Network Topologies

LESSON GOAL:

By the end of the lesson, learners will be able to understand and analyze different types of network topologies and their roles, enabling them to design and configure networks effectively.

RATIONALE:

This lesson focuses on Network Topologies, specifically discussing different types and analyzing their roles. Through analysis, exploration, and practical application, learners will develop the ability to manage networks effectively and justify their choices using technical reasoning. This is Lesson 1 of 5 under sub-topic 2.4.1

PRIOR KNOWLEDGE:

Learners have been introduced to basic computer hardware components and understand that computers can be connected to share resources. They have a basic understanding of what a network is from everyday experience (e.g, mobile data, school Wi-Fi).

REFERENCES:

Computer Science Form 2 Teaching Module, Curriculum Development Centre, Ministry of Education, Zambia, 2026. Topic 2.4: Networking, Sub-Topic 2.4.1: Network Topologies, pp. 27-31.

LEARNING ENVIRONMENT

- **Physical: Natural environment:** Classroom or computer laboratory arranged for group work of learners
- **Virtual:/ Artificial environment:** Not applicable.
- **Technological environment:** Laptops, tablets Optional — computers with Cisco Packet Tracer installed for the extension activity.

TEACHING AND LEARNING MATERIALS / RESOURCES

- Lengths of string or rope (for the kinaesthetic human network activity)
- Printed network topology diagrams: Star, Bus, Ring, Mesh — one set per group
- Scenario cards for three real-world environments (A: school lab, B: rural clinic, C: bank)
- Topology recommendation report template — one per group
- Large paper or whiteboards for diagram drawing
- Standard network diagram symbol reference sheet
- Exercise books and pens for each learner

EXPECTED STANDARD:

Network topologies developed and configured correctly (*This must come from the syllabus as it is*)

LESSON PROGRESSION

STAGES	TEACHER'S ROLE	LEARNERS' ROLE	ASSESSMENT CRITERIA
INTRODUCTION 5 Minutes	Discuss a "hook" with learners. Poses question: "How can one printer be shared by every teacher in the school without anyone leaving their classroom? What happens if the cable to that printer breaks?" Writes key terms on the board: Topology, LAN, Switch, Router, IP Address. • Discussing different types of Network topologies	Listen to the teacher's question. Think independently for 1 minute. Discuss answer with a partner. Share ideas with the class. Copy key terms into exercise books.	Learners accurately and confidently answer the teacher's questions. Learners think independently and critically before responding. Learners actively and collaboratively discuss their answers with a partner. Learners clearly and confidently share ideas with the class. Learners record key terms accurately and neatly.
LESSON DEVELOPMENT 20 minutes	Explains the four main network topologies using printed diagrams: • Star Topology: all devices connect to a central switch. • Bus Topology: all devices share one backbone cable. • Ring Topology: devices connected in a loop. • Mesh Topology: every device	Listen and observe teacher demonstrations. Take structured notes on each topology. Complete a 4-column comparison table (Cost / Reliability / Scalability / Performance) in exercise books. Ask clarification questions.	Learners actively listen and carefully observe teacher demonstrations. Learners take structured and accurate notes on each topology. Learners complete a 4-column comparison table in their exercise books accurately and neatly.

STAGES	TEACHER'S ROLE	LEARNERS' ROLE	ASSESSMENT CRITERIA
	connects to every other device. Introduces the 4-factor trade-off framework: Cost, Reliability, Scalability, Performance. Checks understanding with quick verbal questions after each topology.	Answer the teacher's comprehension checks.	Learners actively and appropriately ask clarification questions to deepen understanding. Learners answer the teacher's comprehension check questions correctly and confidently.
Activity 1: Human Network (Kinaesthetic) 10 Minutes	Directs learners to form a physical "human network" using string as cables: - Star: one learner is the switch in the centre. - Bus: learners in a line passing a message along one string. - Ring: learners form a circle holding adjacent strings. - Mesh: every learner holds strings to multiple others. After each model, shows the corresponding printed topology diagram. Asks: "What happens when this string (cable) breaks?"	Act as nodes (computers) or switches in each topology. Pass a "message" (piece of paper) along the network. Observe the effect when a link is removed. Match each physical model to its printed diagram. Discuss: "Which topology describes our school computer lab?"	Learners actively role-play as nodes or switches in each topology to demonstrate network flow. Learners accurately and effectively pass a message along the network, following the correct sequence and rules. Learners carefully observe the effect when a link is removed and accurately describe the outcome. Learners correctly match each physical model to its corresponding printed diagram. Learners actively discuss and appropriately justify which topology best represents the school computer lab.
Activity 2: Scenario Analysis Task 15 Minutes	Distributes scenario cards to groups of three learners: - A: 30-computer school lab, full-time technician. - B: Rural clinic, 4 computers, limited budget. - C: Bank, 200 workstations, no downtime allowed.	Read and discuss the scenario card in groups. Complete the topology recommendation report: - Which topology is recommended? - What hardware is required? - What are the main risks? - Why is it better than alternatives?	Learners actively and collaboratively read and discuss the scenario card in groups. Learners complete the topology recommendation report accurately and comprehensively, including the recommended topology, required hardware, key

STAGES	TEACHER'S ROLE	LEARNERS' ROLE	ASSESSMENT CRITERIA
	Circulates the classroom and asks probing questions. Prompts groups to justify choices using evidence. Facilitates class debate after presentations.	- Draw a labelled network topology diagram. - Present recommendation to the class and defend choices in debate.	risks, and well-justified reasons for their choice over alternatives. Learners draw a clearly labelled and accurate network topology diagram. Learners confidently and coherently present their recommendations to the class and logically defend their choices during the debate.
EXERCISE 20 Minutes	Reads the exercise scenario aloud: <i>"A small clinic has four computers connected to a single switch. If the cable to one consultation room is unplugged, the other doctors can still work."</i> Clarifies task requirements. Circulates to observe and note learner responses. Collects exercise books at the end.	Work individually and answer: Q1 (Topology Identification): What network topology is the clinic using? Explain why one cable failure does not affect the other computers. Q2 (Configuration and Testing): A new computer is connected to the clinic's switch but cannot access the shared printer. What is one specific step you would take to diagnose the local network connection issue? Write full answers in exercise books.	Learners work independently to correctly identify the network topology used in the clinic scenario. Learners clearly and logically explain why a cable failure does not affect other computers on the network. Learners accurately and systematically explain how to diagnose a local network connection issue when a new computer cannot access the shared printer. Learners write complete, well-structured, and accurate responses in their exercise books
CONCLUSION 10 Minutes	Asks review questions to consolidate learning: "Name the four types of network topology." "What is the main disadvantage of a Bus topology?" "Which topology is most commonly	Answer the teacher's review questions. Contribute to the class summary. Note the homework task in exercise books.	Learners answer the teacher's review questions correctly and confidently Learners actively and meaningfully contribute to the class summary.

STAGES	TEACHER'S ROLE	LEARNERS' ROLE	ASSESSMENT CRITERIA
	used in schools and why?" Summarises key points on the board. Issues homework task.		Learners accurately and neatly record the homework task in their exercise books.
HOMEWORK	Draw a labelled diagram of a Star topology network for an internet cafe with 8 computers. Label all devices (computers, switch, router) and assign IP addresses to each computer using the range 192.168.1.x. In your own words, explain one advantage and one disadvantage of the Star topology for this business.		

LESSON EVALUATION:**Evaluation criteria:**

- Were the specific competences achieved? Provide evidence from learner responses during the exercise.
- Which activities were most effective in engaging learners?
- Which learners need additional support and in which areas?
- What improvements will be made in the next lesson?

Example:

Most learners correctly identified the Star topology in the exercise scenario. A few learners in the lower ability group struggled to justify topology selection and these learners will be given additional scenario practice in the next lesson.